PHARMACY MANAGEMENT SYSTEM

Object-Oriented Programming - Mini Project Report

Submitted in partial fulfillment of the requirements for the
Bachelor of Technology in Computer Science and Engineering

Project Title	Pharmacy Management System
Subject	Object-Oriented Programming (OOP)
Academic Year	2025 - 2026

Team Members

Name	Register Number	Roll No.
Kenneth Martin	240962047	8
Sam Thomas	240962242	18
Mohammed Rayyan Sheik	240962073	12
Vedank Singh	240962043	7

Introduction

Pharmacies handle a surprisingly large amount of data every day. From tracking which medicines are running low to generating bills for dozens of customers, the workload adds up fast. Most small and mid-sized pharmacies still rely on paper registers or basic spreadsheets, which honestly just creates more problems than it solves. Entries get missed, stock counts go wrong, and nobody notices a batch of medicines is about to expire until it's too late.

We built this Pharmacy Management System to fix exactly those kinds of problems. The system is written in Java, with a JavaFX-based desktop interface that any pharmacy staff member can use without much training. It covers the main things a pharmacy needs to keep track of: the medicine stock, customer records, supplier contacts, billing and invoices, and prescriptions from doctors. There's also a background alert system that automatically warns staff when stock is low or medicines are close to their expiry date, without anyone having to manually check.

From an academic standpoint, this project was a chance for us to apply the OOP concepts we've been studying. Encapsulation, inheritance, polymorphism, abstraction, generics, multithreading, serialization, collections - every one of those is used somewhere in this codebase, not just as a checkbox but because it genuinely made the code cleaner and easier to manage. Data is saved between sessions using Java Serialization, and access is controlled by role so admins see everything while pharmacists only get what they need.

Problem Statement

Pharmacy Management System with Inventory Control and Billing Automation

Walk into almost any small pharmacy and you'll likely see someone squinting at a handwritten stock register trying to figure out whether a particular medicine is still available. Billing is done on paper, which means calculation errors are common. Prescriptions get stacked in a drawer and there's no easy way to look one up later. And when a medicine expires on the shelf, nobody really knows until a customer points it out.

These aren't edge cases - they're everyday problems that affect patient safety and pharmacy efficiency. The goal of our project was to build a system that directly addresses these issues. Instead of paper records, everything is stored digitally and can be searched in seconds. Billing is automated so there are no arithmetic mistakes. Background threads continuously monitor stock levels and expiry dates, sending alerts before problems occur. The whole system is designed for the kind of pharmacy that doesn't have a dedicated IT team but still needs reliable software to run smoothly.

Implementation Details

The entire project is built in Java, using JavaFX for the graphical front-end. We organized everything into four packages - model, service, ui, and util - which kept the code from turning into a mess as it grew. There are 32 Java source files in total. The model package holds all the data classes. Service classes handle the actual logic and talk to the data layer. The ui package contains every screen in the application. And util has the shared helper stuff like ID generation and file persistence.

One design decision we made early on was to keep the service layer completely separate from the UI. That way the business logic doesn't get tangled up with button clicks and text fields, and it's much easier to test individual parts. We also made all write operations synchronized since the app runs background threads that could otherwise cause data corruption.

All 32 source files in the project:

- Medicine.java
- MedicineCategory.java
- UserRole.java
- Customer.java
- Supplier.java
- Bill.java
- BillItem.java
- Prescription.java
- PrescriptionItem.java
- User.java
- InventoryManager.java
- BillingService.java
- CustomerService.java
- SupplierService.java
- PrescriptionService.java
- UserService.java
- ExpiryAlertThread.java
- StockAlertThread.java
- BillingThread.java
- BackupService.java
- PharmacyApp.java
- LoginScreen.java
- DashboardScreen.java
- InventoryScreen.java
- BillingScreen.java
- SupplierScreen.java
- CustomerScreen.java
- PrescriptionScreen.java
- ReportsScreen.java
- DataStore.java
- IDGenerator.java
- SampleDataLoader.java

Detailed Explanation of Each File

Model Package (com.pharmacy.model)

1. Medicine.java

This is the core data class for a medicine item in the inventory. It implements both Serializable (so it can be saved to disk) and Comparable<Medicine> (so we can sort lists of medicines by name). We used Integer and Double wrapper classes for the quantity and price fields specifically to show autoboxing and unboxing in action. There are two constructors - one that takes a supplierId and one that doesn't - which covers constructor overloading. The compareTo() method is what powers Collections.sort() when we sort the inventory list.

Code for Medicine.java:

```
package com.pharmacy.model;
import java.io.Serializable;
/**
* Medicine model class representing a medicine in the pharmacy.
* Demonstrates: Encapsulation, Serialization, Constructors, Wrapper classes.
*/
public class Medicine implements Serializable, Comparable<Medicine> {
private static final long serialVersionUID = 1L;
private String medicineId;
private String name;
private String manufacturer;
private MedicineCategory category;
private String batchNumber;
private String expiryDate; // Format: YYYY-MM-DD
private Integer quantity; // Wrapper class demonstration
private Double price; // Wrapper class demonstration
private String supplierId;
// Default constructor
public Medicine() {
}
// Parameterized constructor
public Medicine(String medicineId, String name, String manufacturer,
MedicineCategory category, String batchNumber,
String expiryDate, int quantity, double price, String supplierId) {
this.medicineId = medicineId;
this.name = name;
this.manufacturer = manufacturer;
this.category = category;
this.batchNumber = batchNumber;
this.expiryDate = expiryDate;
this.quantity = quantity; // Autoboxing: int -> Integer
this.price = price; // Autoboxing: double -> Double
this.supplierId = supplierId;
}
// Constructor overloading (without supplier)
public Medicine(String medicineId, String name, String manufacturer,
MedicineCategory category, String batchNumber,
String expiryDate, int quantity, double price) {
this(medicineId, name, manufacturer, category, batchNumber,
expiryDate, quantity, price, "");
}
// Getters and Setters (Encapsulation)
public String getMedicineId() { return medicineId; }
public void setMedicineId(String medicineId) { this.medicineId = medicineId; }
public String getName() { return name; }
public void setName(String name) { this.name = name; }
public String getManufacturer() { return manufacturer; }
public void setManufacturer(String manufacturer) { this.manufacturer = manufacturer; }
public MedicineCategory getCategory() { return category; }
public void setCategory(MedicineCategory category) { this.category = category; }
public String getBatchNumber() { return batchNumber; }
public void setBatchNumber(String batchNumber) { this.batchNumber = batchNumber; }
public String getExpiryDate() { return expiryDate; }
```

```
public void setExpiryDate(String expiryDate) { this.expiryDate = expiryDate; }
public Integer getQuantity() { return quantity; }
public void setQuantity(Integer quantity) { this.quantity = quantity; }
public Double getPrice() { return price; }
public void setPrice(Double price) { this.price = price; }
public String getSupplierId() { return supplierId; }
public void setSupplierId(String supplierId) { this.supplierId = supplierId; }
// Unboxing demonstration
public int getQuantityPrimitive() {
return quantity; // Unboxing: Integer -> int
}
public double getPricePrimitive() {
return price; // Unboxing: Double -> double
}
// Comparable implementation for sorting
@Override
public int compareTo(Medicine other) {
return this.name.compareToIgnoreCase(other.name);
}
@Override
public String toString() {
return medicineId + " | " + name + " | " + category + " | Qty: " + quantity
+ " | Rs." + String.format("%.2f", price) + " | Exp: " + expiryDate;
}
}
```

2. MedicineCategory.java

A simple but useful enum for medicine types. Rather than storing category as a raw String (which is error-prone), we defined a proper enum with values like TABLET, CAPSULE, SYRUP, INJECTION, OINTMENT, DROPS, INHALER, and POWDER. Each value has a display-friendly name string attached. This gets used in the inventory form's dropdown and for filtering the inventory table by category.

Code for MedicineCategory.java:

```
package com.pharmacy.model;
/**
* Enum representing different categories of medicines.
* Demonstrates: Enum usage as required by OSDL syllabus.
*/
public enum MedicineCategory {
TABLET("Tablet"),
CAPSULE("Capsule"),
SYRUP("Syrup"),
INJECTION("Injection"),
OINTMENT("Ointment"),
DROPS("Drops"),
INHALER("Inhaler"),
POWDER("Powder");
private final String displayName;
// Enum constructor
MedicineCategory(String displayName) {
this.displayName = displayName;
}
public String getDisplayName() {
```

```
return displayName;
}
@Override
public String toString() {
return displayName;
}
}
```

3. UserRole.java

Another enum, this one for controlling what a logged-in user is allowed to do. ADMIN gets full access across the system while PHARMACIST has a more limited view. We stored the canManageUsers flag directly inside the enum so the permission check is just a method call rather than a series of if-else comparisons scattered around the codebase.

Code for UserRole.java:

```
package com.pharmacy.model;
/**
* Enum for role-based access control.
* Demonstrates: Enum with methods.
*/
public enum UserRole {
ADMIN("Administrator", true),
PHARMACIST("Pharmacist", false);
private final String displayName;
private final boolean canManageUsers;
UserRole(String displayName, boolean canManageUsers) {
this.displayName = displayName;
this.canManageUsers = canManageUsers;
}
public String getDisplayName() {
return displayName;
}
public boolean canManageUsers() {
return canManageUsers;
}
}
```

4. Customer.java

A straightforward model class for storing patient/customer information. It holds the customer ID, name, phone, email, and address. Implements Serializable so the CustomerService can persist the list to disk. Most of the code here is getters and setters plus a toString() that gives a readable summary line.

Code for Customer.java:

```
package com.pharmacy.model;
import java.io.Serializable;
/**
* Customer/Patient model class.
* Demonstrates: Encapsulation, Serialization.
*/
public class Customer implements Serializable {
private static final long serialVersionUID = 1L;
private String customerId;
```

```
private String name;
private String phone;
private String email;
private String address;
// Default constructor
public Customer() {
}
// Parameterized constructor
public Customer(String customerId, String name, String phone, String email, String address) {
this.customerId = customerId;
this.name = name;
this.phone = phone;
this.email = email;
this.address = address;
}
// Getters and Setters
public String getCustomerId() { return customerId; }
public void setCustomerId(String customerId) { this.customerId = customerId; }
public String getName() { return name; }
public void setName(String name) { this.name = name; }
public String getPhone() { return phone; }
public void setPhone(String phone) { this.phone = phone; }
public String getEmail() { return email; }
public void setEmail(String email) { this.email = email; }
public String getAddress() { return address; }
public void setAddress(String address) { this.address = address; }
@Override
public String toString() {
return customerId + " | " + name + " | " + phone;
}
}
```

5. Supplier.java

Stores details about a medicine supplier company - the company name, a contact person's name, phone number, email, and address. Like all the model classes, it implements Serializable so the data survives application restarts. Supplier records are linked to medicines through the supplierId field in Medicine.java.

Code for Supplier.java:

```
package com.pharmacy.model;
import java.io.Serializable;
/**
* Supplier model class.
* Demonstrates: Encapsulation, Serialization.
*/
public class Supplier implements Serializable {
private static final long serialVersionUID = 1L;
private String supplierId;
private String companyName;
private String contactPerson;
private String phone;
private String email;
private String address;
// Default constructor
```

```
public Supplier() {
}
// Parameterized constructor
public Supplier(String supplierId, String companyName, String contactPerson,
String phone, String email, String address) {
this.supplierId = supplierId;
this.companyName = companyName;
this.contactPerson = contactPerson;
this.phone = phone;
this.email = email;
this.address = address;
}
// Getters and Setters
public String getSupplierId() { return supplierId; }
public void setSupplierId(String supplierId) { this.supplierId = supplierId; }
public String getCompanyName() { return companyName; }
public void setCompanyName(String companyName) { this.companyName = companyName; }
public String getContactPerson() { return contactPerson; }
public void setContactPerson(String contactPerson) { this.contactPerson = contactPerson; }
public String getPhone() { return phone; }
public void setPhone(String phone) { this.phone = phone; }
public String getEmail() { return email; }
public void setEmail(String email) { this.email = email; }
public String getAddress() { return address; }
public void setAddress(String address) { this.address = address; }
@Override
public String toString() {
return supplierId + " | " + companyName + " | " + contactPerson;
}
}
```

6. Bill.java

The billing model was one of the more interesting ones to design. It holds the full details of a transaction: customer info, date, who billed it, and an ArrayList of BillItem objects. The applyDiscount() method is overloaded twice - once that takes a percentage and one that takes an amount plus a boolean flag indicating whether it's a fixed amount or a percentage. That overloading was deliberate to cover the polymorphism requirement.

Code for Bill.java:

```
package com.pharmacy.model;
import java.io.Serializable;
import java.util.ArrayList;
/**
* Bill/Invoice model class for sales transactions.
* Demonstrates: Encapsulation, Serialization, ArrayList, Wrapper classes.
*/
public class Bill implements Serializable {
private static final long serialVersionUID = 1L;
private String billId;
private String customerId;
private String customerName;
private String date;
private ArrayList<BillItem> items;
private Double totalAmount; // Wrapper class
```

```
private Double discount;
private Double netAmount;
private String billedBy; // username of the person who billed
// Default constructor
public Bill() {
this.items = new ArrayList<>();
this.totalAmount = 0.0;
this.discount = 0.0;
this.netAmount = 0.0;
}
// Parameterized constructor
public Bill(String billId, String customerId, String customerName,
String date, String billedBy) {
this.billId = billId;
this.customerId = customerId;
this.customerName = customerName;
this.date = date;
this.billedBy = billedBy;
this.items = new ArrayList<>();
this.totalAmount = 0.0;
this.discount = 0.0;
this.netAmount = 0.0;
}
// Add item and recalculate total
public void addItem(BillItem item) {
items.add(item);
recalculate();
}
// Recalculate totals
public void recalculate() {
double total = 0.0;
for (BillItem item : items) {
total += item.getSubtotal(); // Unboxing
}
this.totalAmount = total; // Autoboxing
this.netAmount = totalAmount - discount;
}
// Method overloading: apply discount by percentage
public void applyDiscount(double percentage) {
this.discount = totalAmount * (percentage / 100.0);
this.netAmount = totalAmount - discount;
}
// Method overloading: apply discount by fixed amount with a flag
public void applyDiscount(double amount, boolean isFixed) {
if (isFixed) {
this.discount = amount;
} else {
this.discount = totalAmount * (amount / 100.0);
}
this.netAmount = totalAmount - discount;
}
// Getters and Setters
public String getBillId() { return billId; }
public void setBillId(String billId) { this.billId = billId; }
public String getCustomerId() { return customerId; }
public void setCustomerId(String customerId) { this.customerId = customerId; }
```

```
public String getCustomerName() { return customerName; }
public void setCustomerName(String customerName) { this.customerName = customerName; }
public String getDate() { return date; }
public void setDate(String date) { this.date = date; }
public ArrayList<BillItem> getItems() { return items; }
public void setItems(ArrayList<BillItem> items) { this.items = items; }
public Double getTotalAmount() { return totalAmount; }
public void setTotalAmount(Double totalAmount) { this.totalAmount = totalAmount; }
public Double getDiscount() { return discount; }
public void setDiscount(Double discount) { this.discount = discount; }
public Double getNetAmount() { return netAmount; }
public void setNetAmount(Double netAmount) { this.netAmount = netAmount; }
public String getBilledBy() { return billedBy; }
public void setBilledBy(String billedBy) { this.billedBy = billedBy; }
@Override
public String toString() {
return billId + " | " + customerName + " | Rs." + String.format("%.2f", netAmount)
+ " | " + date;
}
}
```

7. BillItem.java

Each individual line in a bill is a BillItem. It stores the medicine name, quantity (as Integer - wrapper class), unit price and subtotal (both Double - wrapper class). The subtotal is calculated in the constructor itself so it's always consistent. The autoboxing happens when we assign int/double literals to the Integer/Double fields.

Code for BillItem.java:

```
package com.pharmacy.model;
import java.io.Serializable;
/**
* Individual item in a bill.
* Demonstrates: Encapsulation, Serialization, Wrapper classes.
*/
public class BillItem implements Serializable {
private static final long serialVersionUID = 1L;
private String medicineId;
private String medicineName;
private Integer quantity; // Wrapper class
private Double unitPrice; // Wrapper class
private Double subtotal; // Wrapper class
public BillItem() {
}
public BillItem(String medicineId, String medicineName, int quantity, double unitPrice) {
this.medicineId = medicineId;
this.medicineName = medicineName;
this.quantity = quantity; // Autoboxing
this.unitPrice = unitPrice; // Autoboxing
this.subtotal = quantity * unitPrice; // Autoboxing result
}
// Getters and Setters
public String getMedicineId() { return medicineId; }
public void setMedicineId(String medicineId) { this.medicineId = medicineId; }
```

```
public String getMedicineName() { return medicineName; }
public void setMedicineName(String medicineName) { this.medicineName = medicineName; }
public Integer getQuantity() { return quantity; }
public void setQuantity(Integer quantity) { this.quantity = quantity; }
public Double getUnitPrice() { return unitPrice; }
public void setUnitPrice(Double unitPrice) { this.unitPrice = unitPrice; }
public Double getSubtotal() { return subtotal; }
public void setSubtotal(Double subtotal) { this.subtotal = subtotal; }
@Override
public String toString() {
return medicineName + " x" + quantity + " @ Rs." + String.format("%.2f", unitPrice)
+ " = Rs." + String.format("%.2f", subtotal);
}
}
```

8. Prescription.java

Represents a prescription that a doctor has issued for a patient. It links to a customer via customerId and holds the doctor's name, date, and any notes. The list of prescribed medicines is kept as an ArrayList<PrescriptionItem>. Implements Serializable for persistence through DataStore.

Code for Prescription.java:

```
package com.pharmacy.model;
import java.io.Serializable;
import java.util.ArrayList;
/**
* Prescription model class linking customer to prescribed medicines.
* Demonstrates: Encapsulation, Serialization, ArrayList usage.
*/
public class Prescription implements Serializable {
private static final long serialVersionUID = 1L;
private String prescriptionId;
private String customerId;
private String customerName;
private String doctorName;
private String date;
private ArrayList<PrescriptionItem> items;
private String notes;
// Default constructor
public Prescription() {
this.items = new ArrayList<>();
}
// Parameterized constructor
public Prescription(String prescriptionId, String customerId, String customerName,
String doctorName, String date, String notes) {
this.prescriptionId = prescriptionId;
this.customerId = customerId;
this.customerName = customerName;
this.doctorName = doctorName;
this.date = date;
this.notes = notes;
this.items = new ArrayList<>();
}
// Add item to prescription
public void addItem(PrescriptionItem item) {
```

```
items.add(item);
}
// Getters and Setters
public String getPrescriptionId() { return prescriptionId; }
public void setPrescriptionId(String prescriptionId) { this.prescriptionId = prescriptionId; }
public String getCustomerId() { return customerId; }
public void setCustomerId(String customerId) { this.customerId = customerId; }
public String getCustomerName() { return customerName; }
public void setCustomerName(String customerName) { this.customerName = customerName; }
public String getDoctorName() { return doctorName; }
public void setDoctorName(String doctorName) { this.doctorName = doctorName; }
public String getDate() { return date; }
public void setDate(String date) { this.date = date; }
public ArrayList<PrescriptionItem> getItems() { return items; }
public void setItems(ArrayList<PrescriptionItem> items) { this.items = items; }
public String getNotes() { return notes; }
public void setNotes(String notes) { this.notes = notes; }
@Override
public String toString() {
return prescriptionId + " | Patient: " + customerName + " | Dr. " + doctorName + " | " + date;
}
}
```

9. PrescriptionItem.java

Each line within a prescription is stored as a PrescriptionItem. It records which medicine, how many, and the dosage instructions (like "1-0-1" meaning once in the morning and once at night). Simple class but important for linking prescriptions to actual inventory items.

Code for PrescriptionItem.java:

```
package com.pharmacy.model;
import java.io.Serializable;
/**
* Individual item within a prescription.
* Demonstrates: Encapsulation, Serialization.
*/
public class PrescriptionItem implements Serializable {
private static final long serialVersionUID = 1L;
private String medicineId;
private String medicineName;
private Integer quantity;
private String dosage;
public PrescriptionItem() {
}
public PrescriptionItem(String medicineId, String medicineName, int quantity, String dosage) {
this.medicineId = medicineId;
this.medicineName = medicineName;
this.quantity = quantity; // Autoboxing
this.dosage = dosage;
}
public String getMedicineId() { return medicineId; }
public void setMedicineId(String medicineId) { this.medicineId = medicineId; }
public String getMedicineName() { return medicineName; }
public void setMedicineName(String medicineName) { this.medicineName = medicineName; }
```

```
public Integer getQuantity() { return quantity; }
public void setQuantity(Integer quantity) { this.quantity = quantity; }
public String getDosage() { return dosage; }
public void setDosage(String dosage) { this.dosage = dosage; }
@Override
public String toString() {
return medicineName + " x" + quantity + " (" + dosage + ")";
}
}
```

10. User.java

Represents someone who can log in to the system. Has a username, password, full name, and a UserRole enum. The authenticate() method checks whether a given password matches, which keeps the credential-checking logic inside the model rather than scattered across the login screen. Implements Serializable so user accounts persist.

Code for User.java:

```
package com.pharmacy.model;
import java.io.Serializable;
/**
* User model for login and role-based access.
* Demonstrates: Encapsulation, Enum usage, Serialization.
*/
public class User implements Serializable {
private static final long serialVersionUID = 1L;
private String username;
private String password;
private String fullName;
private UserRole role;
// Default constructor
public User() {
}
// Parameterized constructor
public User(String username, String password, String fullName, UserRole role) {
this.username = username;
this.password = password;
this.fullName = fullName;
this.role = role;
}
// Validate login credentials
public boolean authenticate(String inputPassword) {
return this.password.equals(inputPassword);
}
// Getters and Setters
public String getUsername() { return username; }
public void setUsername(String username) { this.username = username; }
public String getPassword() { return password; }
public void setPassword(String password) { this.password = password; }
public String getFullName() { return fullName; }
public void setFullName(String fullName) { this.fullName = fullName; }
public UserRole getRole() { return role; }
public void setRole(UserRole role) { this.role = role; }
@Override
public String toString() {
```

```
return username + " (" + role.getDisplayName() + ")";
}
}
```

Service Package (com.pharmacy.service)

11. InventoryManager.java

Probably the most used service class in the whole project. It manages the full list of medicines, stored in both an ArrayList (for ordered access) and a HashMap (for instant lookup by ID). Every method that changes data is synchronized since the expiry and stock alert threads also access this data in the background. Deletions use an Iterator to avoid ConcurrentModificationException. Sorting uses Collections.sort() in combination with compareTo() from the Comparable interface, and we added a Comparator-based sort for price and expiry date as well.

Code for InventoryManager.java:

```
package com.pharmacy.service;
import com.pharmacy.model.Medicine;
import com.pharmacy.model.MedicineCategory;
import com.pharmacy.util.DataStore;
import java.util.ArrayList;
import java.util.Collections;
import java.util.Comparator;
import java.util.HashMap;
import java.util.Iterator;
/**
* Manages the medicine inventory.
* Demonstrates: ArrayList, HashMap, Iterator, Collections.sort, Generics, Synchronization.
*/
public class InventoryManager {
private ArrayList<Medicine> medicineList;
private HashMap<String, Medicine> medicineMap; // Maps medicineId -> Medicine object
private DataStore<Medicine> dataStore;
private static final String DATA_FILE = "data/medicines.dat";
private static final String BACKUP_FILE = "data/backup_medicines.txt";
private static final int LOW_STOCK_THRESHOLD = 10;
public InventoryManager() {
this.dataStore = new DataStore<>(DATA_FILE);
this.medicineList = dataStore.loadAll();
this.medicineMap = new HashMap<>();
// Populate HashMap from loaded list
for (Medicine med : medicineList) {
medicineMap.put(med.getMedicineId(), med);
}
}
// Add medicine (synchronized for thread safety)
public synchronized void addMedicine(Medicine medicine) {
medicineList.add(medicine);
medicineMap.put(medicine.getMedicineId(), medicine);
saveData();
}
// Update medicine
public synchronized void updateMedicine(Medicine medicine) {
medicineMap.put(medicine.getMedicineId(), medicine);
// Update in list as well
```

```
for (int i = 0; i < medicineList.size(); i++) {
if (medicineList.get(i).getMedicineId().equals(medicine.getMedicineId())) {
medicineList.set(i, medicine);
break;
}
}
saveData();
}
// Delete medicine using Iterator
public synchronized void deleteMedicine(String medicineId) {
Iterator<Medicine> iterator = medicineList.iterator();
while (iterator.hasNext()) {
Medicine med = iterator.next();
if (med.getMedicineId().equals(medicineId)) {
iterator.remove();
break;
}
}
medicineMap.remove(medicineId);
saveData();
}
// Get medicine by ID using HashMap
public Medicine getMedicineById(String medicineId) {
return medicineMap.get(medicineId);
}
// Search medicines by name
public ArrayList<Medicine> searchByName(String name) {
ArrayList<Medicine> results = new ArrayList<>();
for (Medicine med : medicineList) {
if (med.getName().toLowerCase().contains(name.toLowerCase())) {
results.add(med);
}
}
return results;
}
// Filter by category
public ArrayList<Medicine> filterByCategory(MedicineCategory category) {
ArrayList<Medicine> results = new ArrayList<>();
for (Medicine med : medicineList) {
if (med.getCategory() == category) {
results.add(med);
}
}
return results;
}
// Sort by name using Collections.sort (Comparable)
public ArrayList<Medicine> getSortedByName() {
ArrayList<Medicine> sorted = new ArrayList<>(medicineList);
Collections.sort(sorted); // Uses Medicine's compareTo
return sorted;
}
// Sort by price using Comparator
public ArrayList<Medicine> getSortedByPrice() {
ArrayList<Medicine> sorted = new ArrayList<>(medicineList);
Collections.sort(sorted, new Comparator<Medicine>() {
@Override
```

```
public int compare(Medicine m1, Medicine m2) {
return Double.compare(m1.getPrice(), m2.getPrice());
}
});
return sorted;
}
// Sort by expiry date
public ArrayList<Medicine> getSortedByExpiry() {
ArrayList<Medicine> sorted = new ArrayList<>(medicineList);
Collections.sort(sorted, new Comparator<Medicine>() {
@Override
public int compare(Medicine m1, Medicine m2) {
return m1.getExpiryDate().compareTo(m2.getExpiryDate());
}
});
return sorted;
}
// Get low stock medicines
public ArrayList<Medicine> getLowStockMedicines() {
ArrayList<Medicine> lowStock = new ArrayList<>();
for (Medicine med : medicineList) {
if (med.getQuantity() < LOW_STOCK_THRESHOLD) {
lowStock.add(med);
}
}
return lowStock;
}
// Get expired medicines (compare with current date string YYYY-MM-DD)
public ArrayList<Medicine> getExpiredMedicines(String currentDate) {
ArrayList<Medicine> expired = new ArrayList<>();
for (Medicine med : medicineList) {
if (med.getExpiryDate().compareTo(currentDate) <= 0) {
expired.add(med);
}
}
return expired;
}
// Get medicines expiring soon (within 30 days approximation)
public ArrayList<Medicine> getExpiringSoon(String currentDate, String thresholdDate) {
ArrayList<Medicine> expiring = new ArrayList<>();
for (Medicine med : medicineList) {
if (med.getExpiryDate().compareTo(currentDate) > 0
&& med.getExpiryDate().compareTo(thresholdDate) <= 0) {
expiring.add(med);
}
}
return expiring;
}
// Reduce stock (for billing)
public synchronized boolean reduceStock(String medicineId, int quantity) {
Medicine med = medicineMap.get(medicineId);
if (med != null && med.getQuantity() >= quantity) {
med.setQuantity(med.getQuantity() - quantity);
saveData();
return true;
}
```

```
return false;
}
// Get all medicines
public ArrayList<Medicine> getAllMedicines() {
return medicineList;
}
// Save data
private void saveData() {
dataStore.saveAll(medicineList);
}
// Create backup
public void createBackup() {
dataStore.backupToText(medicineList, BACKUP_FILE);
}
public int getMedicineCount() {
return medicineList.size();
}
}
```

12. BillingService.java

Takes care of everything billing-related: creating new bills, saving them, calculating totals, and writing invoice files to disk. The invoice generation uses FileWriter, which is one of the File I/O requirements. We had a bug early on where the invoice used Unicode box-drawing characters that caused encoding errors on Windows machines, so we switched to plain ASCII borders instead. All public methods are synchronized.

Code for BillingService.java:

```
package com.pharmacy.service;
import com.pharmacy.model.Bill;
import com.pharmacy.model.BillItem;
import com.pharmacy.util.DataStore;
import java.io.FileWriter;
import java.io.IOException;
import java.util.ArrayList;
/**
* Manages billing and invoice operations.
* Demonstrates: ArrayList, Serialization, FileWriter, Method overloading.
*/
public class BillingService {
private ArrayList<Bill> billList;
private DataStore<Bill> dataStore;
private InventoryManager inventoryManager;
private static final String DATA_FILE = "data/bills.dat";
private static final String BACKUP_FILE = "data/backup_bills.txt";
private static final String INVOICE_DIR = "data/invoices/";
public BillingService(InventoryManager inventoryManager) {
this.inventoryManager = inventoryManager;
this.dataStore = new DataStore<>(DATA_FILE);
this.billList = dataStore.loadAll();
}
// Create and save a bill
public synchronized boolean createBill(Bill bill) {
// Reduce stock for each item
for (BillItem item : bill.getItems()) {
```

```
boolean success = inventoryManager.reduceStock(item.getMedicineId(), item.getQuantity());
if (!success) {
return false; // Insufficient stock
}
}
billList.add(bill);
saveData();
generateInvoice(bill);
return true;
}
/**
* Generate invoice as text file.
* Demonstrates: FileWriter usage.
*/
public void generateInvoice(Bill bill) {
java.io.File dir = new java.io.File(INVOICE_DIR);
if (!dir.exists()) {
dir.mkdirs();
}
String invoicePath = INVOICE_DIR + bill.getBillId() + "_invoice.txt";
try (FileWriter writer = new FileWriter(invoicePath)) {
writer.write("■■■
```

```
ArrayList<Bill> results = new ArrayList<>();
for (Bill bill : billList) {
if (bill.getDate().equals(date)) {
results.add(bill);
}
}
return results;
}
// Get total sales
public double getTotalSales() {
double total = 0.0;
for (Bill bill : billList) {
total += bill.getNetAmount();
}
return total;
}
// Get total sales for a date
public double getTotalSalesByDate(String date) {
double total = 0.0;
for (Bill bill : billList) {
if (bill.getDate().equals(date)) {
total += bill.getNetAmount();
}
}
return total;
}
private void saveData() {
dataStore.saveAll(billList);
}
public void createBackup() {
dataStore.backupToText(billList, BACKUP_FILE);
}
public int getBillCount() {
return billList.size();
}
}
```

13. CustomerService.java

Handles the full create, read, update, delete cycle for customer records. Under the hood it keeps an ArrayList for iteration order and a HashMap for fast lookups by ID. When deleting, it uses an Iterator rather than removing inside a for-each loop, which would throw an exception. The DataStore<Customer> instance handles saving everything to disk as a serialized file.

Code for CustomerService.java:

```
package com.pharmacy.service;
import com.pharmacy.model.Customer;
import com.pharmacy.util.DataStore;
import java.util.ArrayList;
import java.util.HashMap;
import java.util.Iterator;
/**
* Manages customer/patient records.
* Demonstrates: ArrayList, HashMap, Iterator, Generics.
*/
```

```
public class CustomerService {
private ArrayList<Customer> customerList;
private HashMap<String, Customer> customerMap;
private DataStore<Customer> dataStore;
private static final String DATA_FILE = "data/customers.dat";
private static final String BACKUP_FILE = "data/backup_customers.txt";
public CustomerService() {
this.dataStore = new DataStore<>(DATA_FILE);
this.customerList = dataStore.loadAll();
this.customerMap = new HashMap<>();
for (Customer cust : customerList) {
customerMap.put(cust.getCustomerId(), cust);
}
}
public synchronized void addCustomer(Customer customer) {
customerList.add(customer);
customerMap.put(customer.getCustomerId(), customer);
saveData();
}
public synchronized void updateCustomer(Customer customer) {
customerMap.put(customer.getCustomerId(), customer);
for (int i = 0; i < customerList.size(); i++) {
if (customerList.get(i).getCustomerId().equals(customer.getCustomerId())) {
customerList.set(i, customer);
break;
}
}
saveData();
}
public synchronized void deleteCustomer(String customerId) {
Iterator<Customer> iterator = customerList.iterator();
while (iterator.hasNext()) {
Customer cust = iterator.next();
if (cust.getCustomerId().equals(customerId)) {
iterator.remove();
break;
}
}
customerMap.remove(customerId);
saveData();
}
public Customer getCustomerById(String customerId) {
return customerMap.get(customerId);
}
public ArrayList<Customer> searchByName(String name) {
ArrayList<Customer> results = new ArrayList<>();
for (Customer cust : customerList) {
if (cust.getName().toLowerCase().contains(name.toLowerCase())) {
results.add(cust);
}
}
return results;
}
public ArrayList<Customer> getAllCustomers() {
return customerList;
```

```
}
private void saveData() {
dataStore.saveAll(customerList);
}
public void createBackup() {
dataStore.backupToText(customerList, BACKUP_FILE);
}
public int getCustomerCount() {
return customerList.size();
}
}
```

14. SupplierService.java

Same structure as CustomerService but for supplier records. CRUD operations backed by ArrayList + HashMap, Iterator used for deletion, and DataStore handles the serialized file storage. Also has a searchByName() method that does a case-insensitive substring search through the supplier list.

Code for SupplierService.java:

```
package com.pharmacy.service;
import com.pharmacy.model.Supplier;
import com.pharmacy.util.DataStore;
import java.util.ArrayList;
import java.util.HashMap;
import java.util.Iterator;
/**
* Manages supplier records.
* Demonstrates: ArrayList, HashMap, Iterator, Generics.
*/
public class SupplierService {
private ArrayList<Supplier> supplierList;
private HashMap<String, Supplier> supplierMap;
private DataStore<Supplier> dataStore;
private static final String DATA_FILE = "data/suppliers.dat";
private static final String BACKUP_FILE = "data/backup_suppliers.txt";
public SupplierService() {
this.dataStore = new DataStore<>(DATA_FILE);
this.supplierList = dataStore.loadAll();
this.supplierMap = new HashMap<>();
for (Supplier sup : supplierList) {
supplierMap.put(sup.getSupplierId(), sup);
}
}
public synchronized void addSupplier(Supplier supplier) {
supplierList.add(supplier);
supplierMap.put(supplier.getSupplierId(), supplier);
saveData();
}
public synchronized void updateSupplier(Supplier supplier) {
supplierMap.put(supplier.getSupplierId(), supplier);
for (int i = 0; i < supplierList.size(); i++) {
if (supplierList.get(i).getSupplierId().equals(supplier.getSupplierId())) {
supplierList.set(i, supplier);
break;
}
```

```
}
saveData();
}
public synchronized void deleteSupplier(String supplierId) {
Iterator<Supplier> iterator = supplierList.iterator();
while (iterator.hasNext()) {
Supplier sup = iterator.next();
if (sup.getSupplierId().equals(supplierId)) {
iterator.remove();
break;
}
}
supplierMap.remove(supplierId);
saveData();
}
public Supplier getSupplierById(String supplierId) {
return supplierMap.get(supplierId);
}
public ArrayList<Supplier> searchByName(String name) {
ArrayList<Supplier> results = new ArrayList<>();
for (Supplier sup : supplierList) {
if (sup.getCompanyName().toLowerCase().contains(name.toLowerCase())) {
results.add(sup);
}
}
return results;
}
public ArrayList<Supplier> getAllSuppliers() {
return supplierList;
}
private void saveData() {
dataStore.saveAll(supplierList);
}
public void createBackup() {
dataStore.backupToText(supplierList, BACKUP_FILE);
}
public int getSupplierCount() {
return supplierList.size();
}
}
```

15. PrescriptionService.java

Manages prescription records. Add and delete operations are supported. The getByCustomerId() method is useful when you want to pull up all prescriptions for a specific patient. Like the other service classes, it delegates persistence entirely to DataStore<Prescription>.

Code for PrescriptionService.java:

```
package com.pharmacy.service;
import com.pharmacy.model.Prescription;
import com.pharmacy.util.DataStore;
import java.util.ArrayList;
import java.util.HashMap;
/**
* Manages prescription records.
```

```
* Demonstrates: ArrayList, HashMap, Generics, Serialization.
*/
public class PrescriptionService {
private ArrayList<Prescription> prescriptionList;
private HashMap<String, Prescription> prescriptionMap;
private DataStore<Prescription> dataStore;
private static final String DATA_FILE = "data/prescriptions.dat";
private static final String BACKUP_FILE = "data/backup_prescriptions.txt";
public PrescriptionService() {
this.dataStore = new DataStore<>(DATA_FILE);
this.prescriptionList = dataStore.loadAll();
this.prescriptionMap = new HashMap<>();
for (Prescription pres : prescriptionList) {
prescriptionMap.put(pres.getPrescriptionId(), pres);
}
}
public synchronized void addPrescription(Prescription prescription) {
prescriptionList.add(prescription);
prescriptionMap.put(prescription.getPrescriptionId(), prescription);
saveData();
}
public synchronized void deletePrescription(String prescriptionId) {
prescriptionList.removeIf(p -> p.getPrescriptionId().equals(prescriptionId));
prescriptionMap.remove(prescriptionId);
saveData();
}
public Prescription getPrescriptionById(String prescriptionId) {
return prescriptionMap.get(prescriptionId);
}
public ArrayList<Prescription> getByCustomerId(String customerId) {
ArrayList<Prescription> results = new ArrayList<>();
for (Prescription pres : prescriptionList) {
if (pres.getCustomerId().equals(customerId)) {
results.add(pres);
}
}
return results;
}
public ArrayList<Prescription> getAllPrescriptions() {
return prescriptionList;
}
private void saveData() {
dataStore.saveAll(prescriptionList);
}
public void createBackup() {
dataStore.backupToText(prescriptionList, BACKUP_FILE);
}
public int getPrescriptionCount() {
return prescriptionList.size();
}
}
```

16. UserService.java

Handles login validation and user management. On the very first run, when no users exist yet, it automatically creates a default admin and a default pharmacist account. The authenticate() method loops through the user list looking for a matching username and then calls User.authenticate() to check the password, keeping the logic clean.

Code for UserService.java:

```
package com.pharmacy.service;
import com.pharmacy.model.User;
import com.pharmacy.model.UserRole;
import com.pharmacy.util.DataStore;
import java.util.ArrayList;
/**
* Manages user authentication and accounts.
* Demonstrates: ArrayList, Serialization, role-based access.
*/
public class UserService {
private ArrayList<User> userList;
private DataStore<User> dataStore;
private User currentUser;
private static final String DATA_FILE = "data/users.dat";
public UserService() {
this.dataStore = new DataStore<>(DATA_FILE);
this.userList = dataStore.loadAll();
// Create default admin if no users exist
if (userList.isEmpty()) {
createDefaultUsers();
}
}
// Create default users for first run
private void createDefaultUsers() {
userList.add(new User("admin", "admin123", "System Administrator", UserRole.ADMIN));
userList.add(new User("pharmacist", "pharma123", "Default Pharmacist", UserRole.PHARMACIST));
dataStore.saveAll(userList);
}
// Authenticate user
public User authenticate(String username, String password) {
for (User user : userList) {
if (user.getUsername().equals(username) && user.authenticate(password)) {
this.currentUser = user;
return user;
}
}
return null;
}
public User getCurrentUser() {
return currentUser;
}
public void setCurrentUser(User user) {
this.currentUser = user;
}
public void addUser(User user) {
userList.add(user);
dataStore.saveAll(userList);
}
```

```
public ArrayList<User> getAllUsers() {
return userList;
}
public void logout() {
this.currentUser = null;
}
}
```

17. ExpiryAlertThread.java

One of two background monitoring threads. This one extends the Thread class directly (the other uses Runnable, so we cover both approaches). It's marked as a daemon thread so it stops automatically when the main application closes. Every 30 seconds it wakes up via sleep(), checks the inventory for expired and about-to-expire medicines, and uses wait()/notify() so other threads can safely read the results.

Code for ExpiryAlertThread.java:

```
package com.pharmacy.service;
import com.pharmacy.model.Medicine;
import java.text.SimpleDateFormat;
import java.util.ArrayList;
import java.util.Calendar;
import java.util.Date;
/**
* Background thread that checks for expired and soon-to-expire medicines.
* Demonstrates: Thread class, sleep(), join(), synchronized, Runnable.
*/
public class ExpiryAlertThread extends Thread {
private InventoryManager inventoryManager;
private volatile boolean running = true;
private ArrayList<Medicine> expiredMedicines;
private ArrayList<Medicine> expiringMedicines;
private final Object lock = new Object();
private AlertListener listener;
// Interface for callback (Abstraction demonstration)
public interface AlertListener {
void onAlertUpdate(ArrayList<Medicine> expired, ArrayList<Medicine> expiringSoon);
}
public ExpiryAlertThread(InventoryManager inventoryManager) {
super("ExpiryAlertThread");
this.inventoryManager = inventoryManager;
this.expiredMedicines = new ArrayList<>();
this.expiringMedicines = new ArrayList<>();
setDaemon(true); // Daemon thread - will stop when main thread stops
}
public void setAlertListener(AlertListener listener) {
this.listener = listener;
}
@Override
public void run() {
while (running) {
try {
checkExpiry();
// Sleep for 30 seconds between checks
Thread.sleep(30000);
```

```
} catch (InterruptedException e) {
Thread.currentThread().interrupt();
break;
}
}
}
// Synchronized method for thread-safe expiry checking
private synchronized void checkExpiry() {
SimpleDateFormat sdf = new SimpleDateFormat("yyyy-MM-dd");
String currentDate = sdf.format(new Date());
// Calculate date 30 days from now
Calendar cal = Calendar.getInstance();
cal.add(Calendar.DAY_OF_MONTH, 30);
String thresholdDate = sdf.format(cal.getTime());
synchronized (lock) {
expiredMedicines = inventoryManager.getExpiredMedicines(currentDate);
expiringMedicines = inventoryManager.getExpiringSoon(currentDate, thresholdDate);
// Notify waiting threads using wait/notify mechanism
lock.notifyAll();
}
// Callback to listener
if (listener != null) {
listener.onAlertUpdate(expiredMedicines, expiringMedicines);
}
}
// Method for other threads to wait for alert data
public ArrayList<Medicine> waitForExpiredList() throws InterruptedException {
synchronized (lock) {
lock.wait(5000); // Wait up to 5 seconds
return new ArrayList<>(expiredMedicines);
}
}
public ArrayList<Medicine> getExpiredMedicines() {
synchronized (lock) {
return new ArrayList<>(expiredMedicines);
}
}
public ArrayList<Medicine> getExpiringMedicines() {
synchronized (lock) {
return new ArrayList<>(expiringMedicines);
}
}
public void stopChecking() {
running = false;
this.interrupt();
}
}
```

18. StockAlertThread.java

The second monitoring thread, this time implementing Runnable rather than extending Thread directly. It checks for medicines with fewer than 10 units in stock and notifies a listener if any are found. A volatile boolean flag named "running" is used to stop the thread cleanly without calling interrupt() or using deprecated Thread.stop(). The checkStock() method is synchronized.

Code for StockAlertThread.java:

```
package com.pharmacy.service;
import com.pharmacy.model.Medicine;
import java.util.ArrayList;
/**
* Background thread that monitors low stock levels.
* Demonstrates: Runnable interface, Thread, sleep(), synchronized.
*/
public class StockAlertThread implements Runnable {
private InventoryManager inventoryManager;
private volatile boolean running = true;
private ArrayList<Medicine> lowStockMedicines;
private AlertListener listener;
// Callback interface (Abstraction)
public interface AlertListener {
void onLowStockUpdate(ArrayList<Medicine> lowStockMedicines);
}
public StockAlertThread(InventoryManager inventoryManager) {
this.inventoryManager = inventoryManager;
this.lowStockMedicines = new ArrayList<>();
}
public void setAlertListener(AlertListener listener) {
this.listener = listener;
}
@Override
public void run() {
while (running) {
try {
checkStock();
Thread.sleep(30000); // Check every 30 seconds
} catch (InterruptedException e) {
Thread.currentThread().interrupt();
break;
}
}
}
private synchronized void checkStock() {
lowStockMedicines = inventoryManager.getLowStockMedicines();
if (listener != null) {
listener.onLowStockUpdate(lowStockMedicines);
}
}
public synchronized ArrayList<Medicine> getLowStockMedicines() {
return new ArrayList<>(lowStockMedicines);
}
public void stopChecking() {
running = false;
}
}
```

19. BillingThread.java

When a bill is ready to be processed, instead of doing everything on the JavaFX application thread (which would freeze the UI), we hand it off to a BillingThread. It extends Thread, sleeps for a bit to simulate processing time, then calls BillingService.createBill() inside a synchronized block. A BillingCallback interface is used to pass the result back to the UI. The UI then uses

Platform.runLater() to update labels and tables safely. We also demonstrated join() by having a separate watcher thread wait for BillingThread to finish.

Code for BillingThread.java:

```
package com.pharmacy.service;
import com.pharmacy.model.Bill;
/**
* Multi-threaded billing simulation.
* Demonstrates: Thread, Runnable, synchronized, sleep(), join().
*
* Simulates billing processing in a separate thread to keep UI responsive.
*/
public class BillingThread extends Thread {
private BillingService billingService;
private Bill bill;
private boolean success;
private String message;
private BillingCallback callback;
// Callback interface for billing result (Abstraction)
public interface BillingCallback {
void onBillingComplete(boolean success, String message, Bill bill);
}
public BillingThread(BillingService billingService, Bill bill, BillingCallback callback) {
super("BillingThread-" + bill.getBillId());
this.billingService = billingService;
this.bill = bill;
this.callback = callback;
this.success = false;
this.message = "";
}
@Override
public void run() {
try {
// Simulate processing time
System.out.println("[" + getName() + "] Processing bill " + bill.getBillId() + "...");
Thread.sleep(1000); // Simulate processing delay
// Process the bill
synchronized (billingService) {
success = billingService.createBill(bill);
}
if (success) {
message = "Bill " + bill.getBillId() + " processed successfully!";
System.out.println("[" + getName() + "] " + message);
} else {
message = "Failed to process bill " + bill.getBillId() + ". Insufficient stock.";
System.out.println("[" + getName() + "] " + message);
}
// Simulate invoice generation time
Thread.sleep(500);
} catch (InterruptedException e) {
message = "Billing interrupted for " + bill.getBillId();
Thread.currentThread().interrupt();
}
// Callback
if (callback != null) {
```

```
callback.onBillingComplete(success, message, bill);
}
}
public boolean isSuccess() {
return success;
}
public String getMessage() {
return message;
}
}
```

20. BackupService.java

Implements Runnable so it can be passed to any Thread or ExecutorService. When run, it calls createBackup() on each of the five service classes in sequence, with short sleep() calls between each one. This writes human-readable text copies of all the data into the data/ directory. A BackupCallback interface lets the UI show a completion message once the backup is done.

Code for BackupService.java:

```
package com.pharmacy.service;
/**
* Backup service that creates file backups of all data.
* Demonstrates: Runnable, Thread, FileWriter (via DataStore).
*/
public class BackupService implements Runnable {
private InventoryManager inventoryManager;
private CustomerService customerService;
private SupplierService supplierService;
private BillingService billingService;
private PrescriptionService prescriptionService;
private BackupCallback callback;
public interface BackupCallback {
void onBackupComplete(String message);
}
public BackupService(InventoryManager inventoryManager, CustomerService customerService,
SupplierService supplierService, BillingService billingService,
PrescriptionService prescriptionService) {
this.inventoryManager = inventoryManager;
this.customerService = customerService;
this.supplierService = supplierService;
this.billingService = billingService;
this.prescriptionService = prescriptionService;
}
public void setCallback(BackupCallback callback) {
this.callback = callback;
}
@Override
public void run() {
try {
System.out.println("[BackupService] Starting backup...");
Thread.sleep(500);
inventoryManager.createBackup();
Thread.sleep(200);
customerService.createBackup();
Thread.sleep(200);
```

```
supplierService.createBackup();
Thread.sleep(200);
billingService.createBackup();
Thread.sleep(200);
prescriptionService.createBackup();
Thread.sleep(200);
String msg = "All data backed up successfully!";
System.out.println("[BackupService] " + msg);
if (callback != null) {
callback.onBackupComplete(msg);
}
} catch (InterruptedException e) {
Thread.currentThread().interrupt();
if (callback != null) {
callback.onBackupComplete("Backup interrupted!");
}
}
}
}
```

UI Package (com.pharmacy.ui)

21. PharmacyApp.java

This is where the application starts. It extends JavaFX's Application class and the start() method sets everything up: all service objects are created, the expiry and stock alert threads are started, sample data is loaded on the first run, and the login screen is shown. Navigation between screens is handled through a showXxxScreen() pattern where each call to one of these methods creates a new scene and passes it to Stage.setScene().

Code for PharmacyApp.java:

```
package com.pharmacy.ui;
import com.pharmacy.model.User;
import com.pharmacy.service.*;
import com.pharmacy.util.SampleDataLoader;
import javafx.application.Application;
import javafx.application.Platform;
import javafx.stage.Stage;
import java.io.File;
/**
* Main JavaFX Application class - Entry point for the Pharmacy Management System.
* Demonstrates: JavaFX Application, Stage management, Inheritance (extends Application).
*/
public class PharmacyApp extends Application {
// Service instances (shared across all screens)
private UserService userService;
private InventoryManager inventoryManager;
private CustomerService customerService;
private SupplierService supplierService;
private BillingService billingService;
private PrescriptionService prescriptionService;
// Alert threads
private ExpiryAlertThread expiryAlertThread;
private Thread stockAlertThreadHandle;
private StockAlertThread stockAlertRunnable;
```

```
private Stage primaryStage;
@Override
public void start(Stage primaryStage) {
this.primaryStage = primaryStage;
// Create data directory
new File("data/invoices").mkdirs();
// Initialize all services
userService = new UserService();
inventoryManager = new InventoryManager();
customerService = new CustomerService();
supplierService = new SupplierService();
billingService = new BillingService(inventoryManager);
prescriptionService = new PrescriptionService();
// Load sample data if empty
SampleDataLoader.loadSampleData(inventoryManager, customerService, supplierService);
// Start background alert threads (Multithreading demonstration)
startAlertThreads();
// Show login screen
showLoginScreen();
primaryStage.setTitle("PharmaCare - Pharmacy Management System");
primaryStage.setWidth(1100);
primaryStage.setHeight(700);
primaryStage.setOnCloseRequest(e -> {
stopAlertThreads();
Platform.exit();
System.exit(0);
});
primaryStage.show();
}
/**
* Start background threads for monitoring.
* Demonstrates: Thread creation (both Thread subclass and Runnable), start(), daemon threads.
*/
private void startAlertThreads() {
// Thread extending Thread class
expiryAlertThread = new ExpiryAlertThread(inventoryManager);
expiryAlertThread.start();
// Thread using Runnable interface
stockAlertRunnable = new StockAlertThread(inventoryManager);
stockAlertThreadHandle = new Thread(stockAlertRunnable, "StockAlertThread");
stockAlertThreadHandle.setDaemon(true);
stockAlertThreadHandle.start();
}
private void stopAlertThreads() {
if (expiryAlertThread != null) {
expiryAlertThread.stopChecking();
}
if (stockAlertRunnable != null) {
stockAlertRunnable.stopChecking();
}
}
// Show login screen
public void showLoginScreen() {
LoginScreen loginScreen = new LoginScreen(this, userService);
primaryStage.setScene(loginScreen.getScene());
```

```
}
// Show dashboard after login
public void showDashboard(User user) {
DashboardScreen dashboard = new DashboardScreen(this, user);
primaryStage.setScene(dashboard.getScene());
}
// Show inventory module
public void showInventoryScreen() {
InventoryScreen screen = new InventoryScreen(this, inventoryManager, supplierService);
primaryStage.setScene(screen.getScene());
}
// Show billing module
public void showBillingScreen() {
BillingScreen screen = new BillingScreen(this, billingService, inventoryManager,
customerService, userService);
primaryStage.setScene(screen.getScene());
}
// Show supplier module
public void showSupplierScreen() {
SupplierScreen screen = new SupplierScreen(this, supplierService);
primaryStage.setScene(screen.getScene());
}
// Show customer module
public void showCustomerScreen() {
CustomerScreen screen = new CustomerScreen(this, customerService);
primaryStage.setScene(screen.getScene());
}
// Show prescription module
public void showPrescriptionScreen() {
PrescriptionScreen screen = new PrescriptionScreen(this, prescriptionService,
customerService, inventoryManager);
primaryStage.setScene(screen.getScene());
}
// Show reports module
public void showReportsScreen() {
ReportsScreen screen = new ReportsScreen(this, inventoryManager, billingService,
customerService, supplierService,
expiryAlertThread, stockAlertRunnable);
primaryStage.setScene(screen.getScene());
}
// Getters for services
public UserService getUserService() { return userService; }
public InventoryManager getInventoryManager() { return inventoryManager; }
public CustomerService getCustomerService() { return customerService; }
public SupplierService getSupplierService() { return supplierService; }
public BillingService getBillingService() { return billingService; }
public PrescriptionService getPrescriptionService() { return prescriptionService; }
public ExpiryAlertThread getExpiryAlertThread() { return expiryAlertThread; }
public StockAlertThread getStockAlertRunnable() { return stockAlertRunnable; }
public Stage getPrimaryStage() { return primaryStage; }
/**
* Main method - entry point.
*/
public static void main(String[] args) {
launch(args);
```

```
}
}
```

22. LoginScreen.java

The first thing users see when they open the app. Built with a GridPane layout, PasswordField to keep the password hidden, and a Label that shows a red error message if login fails. After successful authentication through UserService, the user gets redirected to the dashboard. The role (Admin or Pharmacist) is stored in the User object that comes back from authentication.

Code for LoginScreen.java:

```
package com.pharmacy.ui;
import com.pharmacy.model.User;
import com.pharmacy.service.UserService;
import javafx.geometry.Insets;
import javafx.geometry.Pos;
import javafx.scene.Scene;
import javafx.scene.control.*;
import javafx.scene.layout.*;
import javafx.scene.text.Font;
import javafx.scene.text.FontWeight;
/**
* Login Screen with role-based authentication.
* Demonstrates: JavaFX (GridPane, TextField, Button, Label), Event handling.
*/
public class LoginScreen {
private Scene scene;
private PharmacyApp app;
private UserService userService;
public LoginScreen(PharmacyApp app, UserService userService) {
this.app = app;
this.userService = userService;
createScene();
}
private void createScene() {
// Main layout
VBox mainLayout = new VBox(20);
mainLayout.setAlignment(Pos.CENTER);
mainLayout.setPadding(new Insets(50));
mainLayout.setStyle("-fx-background-color: #2c3e50;");
// Title
Label titleLabel = new Label("PharmaCare");
titleLabel.setFont(Font.font("Arial", FontWeight.BOLD, 36));
titleLabel.setStyle("-fx-text-fill: #ecf0f1;");
Label subtitleLabel = new Label("Pharmacy Management System");
subtitleLabel.setFont(Font.font("Arial", FontWeight.NORMAL, 16));
subtitleLabel.setStyle("-fx-text-fill: #bdc3c7;");
// Login form using GridPane
GridPane grid = new GridPane();
grid.setAlignment(Pos.CENTER);
grid.setHgap(10);
grid.setVgap(15);
grid.setPadding(new Insets(30));
grid.setMaxWidth(400);
grid.setStyle("-fx-background-color: #34495e; -fx-background-radius: 10;");
```

```
Label loginTitle = new Label("Login");
loginTitle.setFont(Font.font("Arial", FontWeight.BOLD, 20));
loginTitle.setStyle("-fx-text-fill: #ecf0f1;");
grid.add(loginTitle, 0, 0, 2, 1);
Label userLabel = new Label("Username:");
userLabel.setStyle("-fx-text-fill: #ecf0f1;");
TextField usernameField = new TextField();
usernameField.setPromptText("Enter username");
usernameField.setPrefWidth(250);
grid.add(userLabel, 0, 1);
grid.add(usernameField, 1, 1);
Label passLabel = new Label("Password:");
passLabel.setStyle("-fx-text-fill: #ecf0f1;");
PasswordField passwordField = new PasswordField();
passwordField.setPromptText("Enter password");
grid.add(passLabel, 0, 2);
grid.add(passwordField, 1, 2);
Label messageLabel = new Label();
messageLabel.setStyle("-fx-text-fill: #e74c3c;");
Button loginBtn = new Button("Login");
loginBtn.setStyle("-fx-background-color: #27ae60; -fx-text-fill: white; " +
"-fx-font-size: 14; -fx-pref-width: 250; -fx-cursor: hand;");
loginBtn.setOnAction(e -> {
String username = usernameField.getText().trim();
String password = passwordField.getText().trim();
if (username.isEmpty() || password.isEmpty()) {
messageLabel.setText("Please enter both username and password.");
return;
}
User user = userService.authenticate(username, password);
if (user != null) {
app.showDashboard(user);
} else {
messageLabel.setText("Invalid credentials! Please try again.");
passwordField.clear();
}
});
// Allow Enter key to login
passwordField.setOnAction(e -> loginBtn.fire());
usernameField.setOnAction(e -> loginBtn.fire());
grid.add(loginBtn, 0, 3, 2, 1);
grid.add(messageLabel, 0, 4, 2, 1);
// Default credentials hint
Label hintLabel = new Label("Default: admin/admin123 or pharmacist/pharma123");
hintLabel.setStyle("-fx-text-fill: #7f8c8d; -fx-font-size: 11;");
mainLayout.getChildren().addAll(titleLabel, subtitleLabel, grid, hintLabel);
scene = new Scene(mainLayout, 1100, 700);
}
public Scene getScene() {
return scene;
}
}
```

23. DashboardScreen.java

After logging in, this is the home screen. It uses a BorderPane as the root, with a dark-colored HBox at the top showing the app name and the logged-in user's details. The center has a GridPane with six buttons, each leading to a different module. There's also a Logout button that returns to the login screen and clears the current user from UserService.

Code for DashboardScreen.java:

```
package com.pharmacy.ui;
import com.pharmacy.model.User;
import javafx.geometry.Insets;
import javafx.geometry.Pos;
import javafx.scene.Scene;
import javafx.scene.control.*;
import javafx.scene.layout.*;
import javafx.scene.text.Font;
import javafx.scene.text.FontWeight;
/**
* Dashboard screen - main navigation hub after login.
* Demonstrates: JavaFX (VBox, HBox, Button, Label, GridPane), Event handling.
*/
public class DashboardScreen {
private Scene scene;
private PharmacyApp app;
private User currentUser;
public DashboardScreen(PharmacyApp app, User currentUser) {
this.app = app;
this.currentUser = currentUser;
createScene();
}
private void createScene() {
BorderPane root = new BorderPane();
root.setStyle("-fx-background-color: #ecf0f1;");
// Top bar
HBox topBar = new HBox(20);
topBar.setPadding(new Insets(15, 25, 15, 25));
topBar.setAlignment(Pos.CENTER_LEFT);
topBar.setStyle("-fx-background-color: #2c3e50;");
Label titleLabel = new Label("PharmaCare Dashboard");
titleLabel.setFont(Font.font("Arial", FontWeight.BOLD, 22));
titleLabel.setStyle("-fx-text-fill: white;");
Region spacer = new Region();
HBox.setHgrow(spacer, Priority.ALWAYS);
Label userLabel = new Label("Welcome, " + currentUser.getFullName()
+ " (" + currentUser.getRole().getDisplayName() + ")");
userLabel.setStyle("-fx-text-fill: #bdc3c7; -fx-font-size: 13;");
Button logoutBtn = new Button("Logout");
logoutBtn.setStyle("-fx-background-color: #e74c3c; -fx-text-fill: white; -fx-cursor: hand;");
logoutBtn.setOnAction(e -> {
app.getUserService().logout();
app.showLoginScreen();
});
topBar.getChildren().addAll(titleLabel, spacer, userLabel, logoutBtn);
root.setTop(topBar);
// Center - Module buttons in a grid
GridPane moduleGrid = new GridPane();
```

```
moduleGrid.setAlignment(Pos.CENTER);
moduleGrid.setHgap(25);
moduleGrid.setVgap(25);
moduleGrid.setPadding(new Insets(40));
// Module buttons
Button inventoryBtn = createModuleButton("Inventory\nManagement",
"Manage medicines, stock,\nbatch & expiry tracking", "#3498db");
inventoryBtn.setOnAction(e -> app.showInventoryScreen());
Button billingBtn = createModuleButton("Sales &\nBilling",
"Create bills, generate\ninvoices for customers", "#27ae60");
billingBtn.setOnAction(e -> app.showBillingScreen());
Button supplierBtn = createModuleButton("Supplier\nManagement",
"Manage supplier records\nand contact details", "#e67e22");
supplierBtn.setOnAction(e -> app.showSupplierScreen());
Button customerBtn = createModuleButton("Customer\nRecords",
"Manage patient/customer\ninformation", "#9b59b6");
customerBtn.setOnAction(e -> app.showCustomerScreen());
Button prescriptionBtn = createModuleButton("Prescription\nHandling",
"Record and manage\nprescription details", "#1abc9c");
prescriptionBtn.setOnAction(e -> app.showPrescriptionScreen());
Button reportsBtn = createModuleButton("Reports &\nAlerts",
"View sales reports, stock\n& expiry alerts", "#e74c3c");
reportsBtn.setOnAction(e -> app.showReportsScreen());
// Arrange in 3x2 grid
moduleGrid.add(inventoryBtn, 0, 0);
moduleGrid.add(billingBtn, 1, 0);
moduleGrid.add(supplierBtn, 2, 0);
moduleGrid.add(customerBtn, 0, 1);
moduleGrid.add(prescriptionBtn, 1, 1);
moduleGrid.add(reportsBtn, 2, 1);
root.setCenter(moduleGrid);
// Bottom status bar
HBox statusBar = new HBox(20);
statusBar.setPadding(new Insets(10, 25, 10, 25));
statusBar.setStyle("-fx-background-color: #34495e;");
Label statusLabel = new Label("Medicines: " + app.getInventoryManager().getMedicineCount()
+ " | Customers: " + app.getCustomerService().getCustomerCount()
+ " | Suppliers: " + app.getSupplierService().getSupplierCount()
+ " | Bills: " + app.getBillingService().getBillCount());
statusLabel.setStyle("-fx-text-fill: #bdc3c7; -fx-font-size: 12;");
statusBar.getChildren().add(statusLabel);
root.setBottom(statusBar);
scene = new Scene(root, 1100, 700);
}
// Helper to create styled module buttons
private Button createModuleButton(String title, String description, String color) {
VBox content = new VBox(8);
content.setAlignment(Pos.CENTER);
Label titleLbl = new Label(title);
titleLbl.setFont(Font.font("Arial", FontWeight.BOLD, 16));
titleLbl.setStyle("-fx-text-fill: white; -fx-text-alignment: center;");
Label descLbl = new Label(description);
descLbl.setStyle("-fx-text-fill: #ecf0f1; -fx-font-size: 11; -fx-text-alignment: center;");
content.getChildren().addAll(titleLbl, descLbl);
```

```
Button btn = new Button();
btn.setGraphic(content);
btn.setPrefSize(230, 130);
btn.setStyle("-fx-background-color: " + color + "; -fx-background-radius: 10; -fx-cursor:
hand;");
return btn;
}
public Scene getScene() {
return scene;
}
}
```

24. InventoryScreen.java

The most feature-rich screen in the system. Shows all medicines in a TableView. On the left side there's a form for adding and editing medicines, with a ComboBox for selecting the medicine category (populated from the MedicineCategory enum). There's a search box, a sort dropdown, and a category filter at the top of the table. Clicking a row auto-fills the form fields for editing.

Code for InventoryScreen.java:

```
package com.pharmacy.ui;
import com.pharmacy.model.Medicine;
import com.pharmacy.model.MedicineCategory;
import com.pharmacy.service.InventoryManager;
import com.pharmacy.service.SupplierService;
import com.pharmacy.util.IDGenerator;
import javafx.collections.FXCollections;
import javafx.collections.ObservableList;
import javafx.geometry.Insets;
import javafx.geometry.Pos;
import javafx.scene.Scene;
import javafx.scene.control.*;
import javafx.scene.control.cell.PropertyValueFactory;
import javafx.scene.layout.*;
import javafx.scene.text.Font;
import javafx.scene.text.FontWeight;
import java.util.ArrayList;
/**
* Inventory Management Screen.
* Demonstrates: JavaFX TableView, GridPane, VBox/HBox, Buttons, ComboBox.
*/
public class InventoryScreen {
private Scene scene;
private PharmacyApp app;
private InventoryManager inventoryManager;
private SupplierService supplierService;
private TableView<Medicine> tableView;
private ObservableList<Medicine> tableData;
public InventoryScreen(PharmacyApp app, InventoryManager inventoryManager,
SupplierService supplierService) {
this.app = app;
this.inventoryManager = inventoryManager;
this.supplierService = supplierService;
createScene();
}
@SuppressWarnings("unchecked")
```

```
private void createScene() {
BorderPane root = new BorderPane();
root.setStyle("-fx-background-color: #ecf0f1;");
// Top bar
HBox topBar = createTopBar("Inventory Management");
root.setTop(topBar);
// Left side - Form
VBox formBox = createForm();
root.setLeft(formBox);
// Center - Table
VBox tableBox = createTable();
root.setCenter(tableBox);
scene = new Scene(root, 1100, 700);
}
private HBox createTopBar(String title) {
HBox topBar = new HBox(15);
topBar.setPadding(new Insets(12, 20, 12, 20));
topBar.setAlignment(Pos.CENTER_LEFT);
topBar.setStyle("-fx-background-color: #2c3e50;");
Button backBtn = new Button("< Back");
backBtn.setStyle("-fx-background-color: #7f8c8d; -fx-text-fill: white; -fx-cursor: hand;");
backBtn.setOnAction(e -> app.showDashboard(app.getUserService().getCurrentUser()));
Label titleLabel = new Label(title);
titleLabel.setFont(Font.font("Arial", FontWeight.BOLD, 20));
titleLabel.setStyle("-fx-text-fill: white;");
topBar.getChildren().addAll(backBtn, titleLabel);
return topBar;
}
private VBox createForm() {
VBox formBox = new VBox(10);
formBox.setPadding(new Insets(15));
formBox.setPrefWidth(320);
formBox.setStyle("-fx-background-color: white; -fx-border-color: #bdc3c7; -fx-border-width: 0 1
0 0;");
Label formTitle = new Label("Add / Edit Medicine");
formTitle.setFont(Font.font("Arial", FontWeight.BOLD, 15));
// Form fields using GridPane
GridPane grid = new GridPane();
grid.setVgap(8);
grid.setHgap(8);
TextField idField = new TextField();
idField.setEditable(false);
idField.setPromptText("Auto-generated");
TextField nameField = new TextField();
nameField.setPromptText("Medicine name");
TextField mfgField = new TextField();
mfgField.setPromptText("Manufacturer");
ComboBox<MedicineCategory> categoryBox = new ComboBox<>();
categoryBox.getItems().addAll(MedicineCategory.values());
categoryBox.setPromptText("Select Category");
categoryBox.setPrefWidth(180);
TextField batchField = new TextField();
batchField.setPromptText("Batch number");
TextField expiryField = new TextField();
```

```
expiryField.setPromptText("YYYY-MM-DD");
TextField qtyField = new TextField();
qtyField.setPromptText("Quantity");
TextField priceField = new TextField();
priceField.setPromptText("Price (Rs.)");
TextField supplierField = new TextField();
supplierField.setPromptText("Supplier ID");
int row = 0;
grid.add(new Label("ID:"), 0, row); grid.add(idField, 1, row++);
grid.add(new Label("Name:"), 0, row); grid.add(nameField, 1, row++);
grid.add(new Label("Manufacturer:"), 0, row); grid.add(mfgField, 1, row++);
grid.add(new Label("Category:"), 0, row); grid.add(categoryBox, 1, row++);
grid.add(new Label("Batch No:"), 0, row); grid.add(batchField, 1, row++);
grid.add(new Label("Expiry Date:"), 0, row); grid.add(expiryField, 1, row++);
grid.add(new Label("Quantity:"), 0, row); grid.add(qtyField, 1, row++);
grid.add(new Label("Price:"), 0, row); grid.add(priceField, 1, row++);
grid.add(new Label("Supplier ID:"), 0, row); grid.add(supplierField, 1, row++);
Label statusLabel = new Label();
statusLabel.setStyle("-fx-text-fill: #27ae60;");
// Buttons
HBox btnBox = new HBox(8);
Button addBtn = new Button("Add");
addBtn.setStyle("-fx-background-color: #27ae60; -fx-text-fill: white; -fx-cursor: hand;");
Button updateBtn = new Button("Update");
updateBtn.setStyle("-fx-background-color: #2980b9; -fx-text-fill: white; -fx-cursor: hand;");
Button deleteBtn = new Button("Delete");
deleteBtn.setStyle("-fx-background-color: #e74c3c; -fx-text-fill: white; -fx-cursor: hand;");
Button clearBtn = new Button("Clear");
clearBtn.setStyle("-fx-background-color: #7f8c8d; -fx-text-fill: white; -fx-cursor: hand;");
btnBox.getChildren().addAll(addBtn, updateBtn, deleteBtn, clearBtn);
// Add button action
addBtn.setOnAction(e -> {
try {
String id = IDGenerator.generateMedicineId();
Medicine med = new Medicine(id, nameField.getText(), mfgField.getText(),
categoryBox.getValue(), batchField.getText(), expiryField.getText(),
Integer.parseInt(qtyField.getText()), // Wrapper class parsing
Double.parseDouble(priceField.getText()), // Wrapper class parsing
supplierField.getText());
inventoryManager.addMedicine(med);
refreshTable();
clearFields(idField, nameField, mfgField, categoryBox, batchField,
expiryField, qtyField, priceField, supplierField);
statusLabel.setText("Medicine added: " + id);
statusLabel.setStyle("-fx-text-fill: #27ae60;");
} catch (Exception ex) {
statusLabel.setText("Error: " + ex.getMessage());
statusLabel.setStyle("-fx-text-fill: #e74c3c;");
}
});
// Update button action
updateBtn.setOnAction(e -> {
try {
String id = idField.getText();
if (id.isEmpty()) {
statusLabel.setText("Select a medicine first!");
```

```
statusLabel.setStyle("-fx-text-fill: #e74c3c;");
return;
}
Medicine med = new Medicine(id, nameField.getText(), mfgField.getText(),
categoryBox.getValue(), batchField.getText(), expiryField.getText(),
Integer.parseInt(qtyField.getText()),
Double.parseDouble(priceField.getText()),
supplierField.getText());
inventoryManager.updateMedicine(med);
refreshTable();
statusLabel.setText("Medicine updated: " + id);
statusLabel.setStyle("-fx-text-fill: #27ae60;");
} catch (Exception ex) {
statusLabel.setText("Error: " + ex.getMessage());
statusLabel.setStyle("-fx-text-fill: #e74c3c;");
}
});
// Delete button action
deleteBtn.setOnAction(e -> {
String id = idField.getText();
if (!id.isEmpty()) {
inventoryManager.deleteMedicine(id);
refreshTable();
clearFields(idField, nameField, mfgField, categoryBox, batchField,
expiryField, qtyField, priceField, supplierField);
statusLabel.setText("Medicine deleted: " + id);
}
});
// Clear button
clearBtn.setOnAction(e -> {
clearFields(idField, nameField, mfgField, categoryBox, batchField,
expiryField, qtyField, priceField, supplierField);
statusLabel.setText("");
});
// Table selection listener - populate form
tableView = new TableView<>();
tableView.getSelectionModel().selectedItemProperty().addListener((obs, oldVal, newVal) -> {
if (newVal != null) {
idField.setText(newVal.getMedicineId());
nameField.setText(newVal.getName());
mfgField.setText(newVal.getManufacturer());
categoryBox.setValue(newVal.getCategory());
batchField.setText(newVal.getBatchNumber());
expiryField.setText(newVal.getExpiryDate());
qtyField.setText(String.valueOf(newVal.getQuantity()));
priceField.setText(String.valueOf(newVal.getPrice()));
supplierField.setText(newVal.getSupplierId());
}
});
formBox.getChildren().addAll(formTitle, grid, btnBox, statusLabel);
return formBox;
}
@SuppressWarnings("unchecked")
private VBox createTable() {
VBox tableBox = new VBox(10);
tableBox.setPadding(new Insets(15));
```

```
// Search and sort controls
HBox controlsBox = new HBox(10);
controlsBox.setAlignment(Pos.CENTER_LEFT);
TextField searchField = new TextField();
searchField.setPromptText("Search by name...");
searchField.setPrefWidth(200);
Button searchBtn = new Button("Search");
searchBtn.setStyle("-fx-background-color: #3498db; -fx-text-fill: white; -fx-cursor: hand;");
searchBtn.setOnAction(e -> {
String query = searchField.getText().trim();
if (query.isEmpty()) {
refreshTable();
} else {
ArrayList<Medicine> results = inventoryManager.searchByName(query);
tableData = FXCollections.observableArrayList(results);
tableView.setItems(tableData);
}
});
ComboBox<String> sortBox = new ComboBox<>();
sortBox.getItems().addAll("Sort by Name", "Sort by Price", "Sort by Expiry");
sortBox.setPromptText("Sort by...");
sortBox.setOnAction(e -> {
String selected = sortBox.getValue();
ArrayList<Medicine> sorted;
if ("Sort by Name".equals(selected)) {
sorted = inventoryManager.getSortedByName();
} else if ("Sort by Price".equals(selected)) {
sorted = inventoryManager.getSortedByPrice();
} else {
sorted = inventoryManager.getSortedByExpiry();
}
tableData = FXCollections.observableArrayList(sorted);
tableView.setItems(tableData);
});
ComboBox<MedicineCategory> filterBox = new ComboBox<>();
filterBox.getItems().addAll(MedicineCategory.values());
filterBox.setPromptText("Filter by Category");
filterBox.setOnAction(e -> {
MedicineCategory cat = filterBox.getValue();
if (cat != null) {
ArrayList<Medicine> filtered = inventoryManager.filterByCategory(cat);
tableData = FXCollections.observableArrayList(filtered);
tableView.setItems(tableData);
}
});
Button refreshBtn = new Button("Show All");
refreshBtn.setStyle("-fx-background-color: #7f8c8d; -fx-text-fill: white; -fx-cursor: hand;");
refreshBtn.setOnAction(e -> refreshTable());
controlsBox.getChildren().addAll(searchField, searchBtn, sortBox, filterBox, refreshBtn);
// Table columns
TableColumn<Medicine, String> idCol = new TableColumn<>("ID");
idCol.setCellValueFactory(new PropertyValueFactory<>("medicineId"));
idCol.setPrefWidth(80);
TableColumn<Medicine, String> nameCol = new TableColumn<>("Name");
nameCol.setCellValueFactory(new PropertyValueFactory<>("name"));
nameCol.setPrefWidth(150);
```

```
TableColumn<Medicine, MedicineCategory> catCol = new TableColumn<>("Category");
catCol.setCellValueFactory(new PropertyValueFactory<>("category"));
catCol.setPrefWidth(80);
TableColumn<Medicine, String> batchCol = new TableColumn<>("Batch");
batchCol.setCellValueFactory(new PropertyValueFactory<>("batchNumber"));
batchCol.setPrefWidth(70);
TableColumn<Medicine, String> expiryCol = new TableColumn<>("Expiry");
expiryCol.setCellValueFactory(new PropertyValueFactory<>("expiryDate"));
expiryCol.setPrefWidth(90);
TableColumn<Medicine, Integer> qtyCol = new TableColumn<>("Qty");
qtyCol.setCellValueFactory(new PropertyValueFactory<>("quantity"));
qtyCol.setPrefWidth(50);
TableColumn<Medicine, Double> priceCol = new TableColumn<>("Price");
priceCol.setCellValueFactory(new PropertyValueFactory<>("price"));
priceCol.setPrefWidth(70);
tableView.getColumns().addAll(idCol, nameCol, catCol, batchCol, expiryCol, qtyCol, priceCol);
refreshTable();
VBox.setVgrow(tableView, Priority.ALWAYS);
tableBox.getChildren().addAll(controlsBox, tableView);
return tableBox;
}
private void refreshTable() {
tableData = FXCollections.observableArrayList(inventoryManager.getAllMedicines());
tableView.setItems(tableData);
}
private void clearFields(TextField idField, TextField nameField, TextField mfgField,
ComboBox<MedicineCategory> categoryBox, TextField batchField,
TextField expiryField, TextField qtyField, TextField priceField,
TextField supplierField) {
idField.clear();
nameField.clear();
mfgField.clear();
categoryBox.setValue(null);
batchField.clear();
expiryField.clear();
qtyField.clear();
priceField.clear();
supplierField.clear();
tableView.getSelectionModel().clearSelection();
}
public Scene getScene() {
return scene;
}
}
```

25. BillingScreen.java

The billing screen is split into two sections. On the left you build the bill: enter a customer ID (auto-fills the name), add medicines by ID and quantity, optionally apply a discount percentage, then hit Generate Bill. The processing runs in a BillingThread so the UI stays responsive. On the right, current bill items are shown in one table and past bills in another.

Code for BillingScreen.java:

```
package com.pharmacy.ui;
import com.pharmacy.model.*;
```

```
import com.pharmacy.service.*;
import com.pharmacy.util.IDGenerator;
import javafx.application.Platform;
import javafx.collections.FXCollections;
import javafx.collections.ObservableList;
import javafx.geometry.Insets;
import javafx.geometry.Pos;
import javafx.scene.Scene;
import javafx.scene.control.*;
import javafx.scene.control.cell.PropertyValueFactory;
import javafx.scene.layout.*;
import javafx.scene.text.Font;
import javafx.scene.text.FontWeight;
import java.text.SimpleDateFormat;
import java.util.Date;
/**
* Sales and Billing Screen with multi-threaded billing simulation.
* Demonstrates: JavaFX TableView, GridPane, VBox/HBox, Multithreaded billing.
*/
public class BillingScreen {
private Scene scene;
private PharmacyApp app;
private BillingService billingService;
private InventoryManager inventoryManager;
private CustomerService customerService;
private UserService userService;
private TableView<BillItem> billItemsTable;
private ObservableList<BillItem> billItems;
private Label totalLabel;
private Label netLabel;
private Bill currentBill;
public BillingScreen(PharmacyApp app, BillingService billingService,
InventoryManager inventoryManager, CustomerService customerService,
UserService userService) {
this.app = app;
this.billingService = billingService;
this.inventoryManager = inventoryManager;
this.customerService = customerService;
this.userService = userService;
this.billItems = FXCollections.observableArrayList();
createScene();
}
@SuppressWarnings("unchecked")
private void createScene() {
BorderPane root = new BorderPane();
root.setStyle("-fx-background-color: #ecf0f1;");
// Top bar
HBox topBar = new HBox(15);
topBar.setPadding(new Insets(12, 20, 12, 20));
topBar.setAlignment(Pos.CENTER_LEFT);
topBar.setStyle("-fx-background-color: #2c3e50;");
Button backBtn = new Button("< Back");
backBtn.setStyle("-fx-background-color: #7f8c8d; -fx-text-fill: white; -fx-cursor: hand;");
backBtn.setOnAction(e -> app.showDashboard(userService.getCurrentUser()));
Label titleLabel = new Label("Sales & Billing");
```

```
titleLabel.setFont(Font.font("Arial", FontWeight.BOLD, 20));
titleLabel.setStyle("-fx-text-fill: white;");
topBar.getChildren().addAll(backBtn, titleLabel);
root.setTop(topBar);
// Left - Bill form
VBox formBox = new VBox(12);
formBox.setPadding(new Insets(15));
formBox.setPrefWidth(320);
formBox.setStyle("-fx-background-color: white; -fx-border-color: #bdc3c7; -fx-border-width: 0 1
0 0;");
Label formTitle = new Label("New Bill");
formTitle.setFont(Font.font("Arial", FontWeight.BOLD, 15));
GridPane custGrid = new GridPane();
custGrid.setVgap(8);
custGrid.setHgap(8);
TextField custIdField = new TextField();
custIdField.setPromptText("Customer ID");
TextField custNameField = new TextField();
custNameField.setPromptText("Customer Name");
// Auto-fill customer name when ID is entered
custIdField.textProperty().addListener((obs, oldVal, newVal) -> {
if (newVal != null && !newVal.isEmpty()) {
Customer cust = customerService.getCustomerById(newVal.trim());
if (cust != null) {
custNameField.setText(cust.getName());
}
}
});
custGrid.add(new Label("Customer ID:"), 0, 0);
custGrid.add(custIdField, 1, 0);
custGrid.add(new Label("Customer Name:"), 0, 1);
custGrid.add(custNameField, 1, 1);
// Add medicine to bill
Label addItemTitle = new Label("Add Medicine to Bill");
addItemTitle.setFont(Font.font("Arial", FontWeight.BOLD, 13));
GridPane itemGrid = new GridPane();
itemGrid.setVgap(8);
itemGrid.setHgap(8);
TextField medIdField = new TextField();
medIdField.setPromptText("Medicine ID");
Label medNameLabel = new Label("-");
Label medPriceLabel = new Label("-");
Label medStockLabel = new Label("-");
TextField medQtyField = new TextField();
medQtyField.setPromptText("Quantity");
// Auto-fill medicine details
medIdField.textProperty().addListener((obs, oldVal, newVal) -> {
if (newVal != null && !newVal.isEmpty()) {
Medicine med = inventoryManager.getMedicineById(newVal.trim());
if (med != null) {
medNameLabel.setText(med.getName());
medPriceLabel.setText("Rs." + String.format("%.2f", med.getPrice()));
medStockLabel.setText("Stock: " + med.getQuantity());
}
}
```

```
});
int r = 0;
itemGrid.add(new Label("Medicine ID:"), 0, r); itemGrid.add(medIdField, 1, r++);
itemGrid.add(new Label("Name:"), 0, r); itemGrid.add(medNameLabel, 1, r++);
itemGrid.add(new Label("Price:"), 0, r); itemGrid.add(medPriceLabel, 1, r++);
itemGrid.add(new Label("Available:"), 0, r); itemGrid.add(medStockLabel, 1, r++);
itemGrid.add(new Label("Quantity:"), 0, r); itemGrid.add(medQtyField, 1, r++);
Label statusLabel = new Label();
Button addItemBtn = new Button("Add to Bill");
addItemBtn.setStyle("-fx-background-color: #3498db; -fx-text-fill: white; -fx-cursor: hand;
-fx-pref-width: 200;");
addItemBtn.setOnAction(e -> {
try {
String medId = medIdField.getText().trim();
Medicine med = inventoryManager.getMedicineById(medId);
if (med == null) {
statusLabel.setText("Medicine not found!");
statusLabel.setStyle("-fx-text-fill: #e74c3c;");
return;
}
int qty = Integer.parseInt(medQtyField.getText().trim());
if (qty > med.getQuantity()) {
statusLabel.setText("Insufficient stock!");
statusLabel.setStyle("-fx-text-fill: #e74c3c;");
return;
}
BillItem item = new BillItem(medId, med.getName(), qty, med.getPrice());
billItems.add(item);
updateTotals();
medIdField.clear();
medQtyField.clear();
medNameLabel.setText("-");
medPriceLabel.setText("-");
medStockLabel.setText("-");
statusLabel.setText("Item added!");
statusLabel.setStyle("-fx-text-fill: #27ae60;");
} catch (NumberFormatException ex) {
statusLabel.setText("Invalid quantity!");
statusLabel.setStyle("-fx-text-fill: #e74c3c;");
}
});
// Totals
totalLabel = new Label("Total: Rs.0.00");
totalLabel.setFont(Font.font("Arial", FontWeight.BOLD, 14));
TextField discountField = new TextField("0");
discountField.setPromptText("Discount %");
discountField.setPrefWidth(80);
netLabel = new Label("Net: Rs.0.00");
netLabel.setFont(Font.font("Arial", FontWeight.BOLD, 16));
netLabel.setStyle("-fx-text-fill: #27ae60;");
HBox totalsBox = new HBox(10);
totalsBox.setAlignment(Pos.CENTER_LEFT);
totalsBox.getChildren().addAll(new Label("Discount %:"), discountField);
// Generate Bill button - uses multithreaded billing
Button generateBtn = new Button("Generate Bill (Threaded)");
generateBtn.setStyle("-fx-background-color: #27ae60; -fx-text-fill: white; " +
```

```
"-fx-font-size: 14; -fx-pref-width: 280; -fx-cursor: hand;");
generateBtn.setOnAction(e -> {
if (billItems.isEmpty()) {
statusLabel.setText("Add items first!");
statusLabel.setStyle("-fx-text-fill: #e74c3c;");
return;
}
if (custNameField.getText().trim().isEmpty()) {
statusLabel.setText("Enter customer name!");
statusLabel.setStyle("-fx-text-fill: #e74c3c;");
return;
}
String billId = IDGenerator.generateBillId();
String date = new SimpleDateFormat("yyyy-MM-dd").format(new Date());
currentBill = new Bill(billId, custIdField.getText().trim(),
custNameField.getText().trim(), date,
userService.getCurrentUser().getUsername());
for (BillItem item : billItems) {
currentBill.addItem(item);
}
try {
double discPct = Double.parseDouble(discountField.getText().trim());
currentBill.applyDiscount(discPct); // Method overloading demo
} catch (NumberFormatException ex) {
// No discount
}
// Multi-threaded billing using BillingThread
statusLabel.setText("Processing bill...");
statusLabel.setStyle("-fx-text-fill: #3498db;");
generateBtn.setDisable(true);
BillingThread billingThread = new BillingThread(billingService, currentBill,
(success, message, bill) -> {
// Use Platform.runLater to update UI from thread
Platform.runLater(() -> {
if (success) {
statusLabel.setText(message + " Invoice generated!");
statusLabel.setStyle("-fx-text-fill: #27ae60;");
billItems.clear();
updateTotals();
custIdField.clear();
custNameField.clear();
} else {
statusLabel.setText(message);
statusLabel.setStyle("-fx-text-fill: #e74c3c;");
}
generateBtn.setDisable(false);
});
});
billingThread.start();
// Demonstrate join() - wait for billing thread in another thread
Thread waitThread = new Thread(() -> {
try {
billingThread.join(); // Wait for billing to complete
System.out.println("Billing thread completed via join()");
} catch (InterruptedException ex) {
Thread.currentThread().interrupt();
```

```
}
});
waitThread.setDaemon(true);
waitThread.start();
});
Button clearBillBtn = new Button("Clear Bill");
clearBillBtn.setStyle("-fx-background-color: #e74c3c; -fx-text-fill: white; -fx-cursor:
hand;");
clearBillBtn.setOnAction(e -> {
billItems.clear();
updateTotals();
statusLabel.setText("");
});
formBox.getChildren().addAll(formTitle, custGrid,
new Separator(), addItemTitle, itemGrid, addItemBtn, statusLabel,
new Separator(), totalLabel, totalsBox, netLabel,
generateBtn, clearBillBtn);
root.setLeft(formBox);
// Center - Bill items table
VBox tableBox = new VBox(10);
tableBox.setPadding(new Insets(15));
Label tableTitle = new Label("Current Bill Items");
tableTitle.setFont(Font.font("Arial", FontWeight.BOLD, 15));
billItemsTable = new TableView<>();
billItemsTable.setItems(billItems);
TableColumn<BillItem, String> nameCol = new TableColumn<>("Medicine");
nameCol.setCellValueFactory(new PropertyValueFactory<>("medicineName"));
nameCol.setPrefWidth(200);
TableColumn<BillItem, Integer> qtyCol = new TableColumn<>("Qty");
qtyCol.setCellValueFactory(new PropertyValueFactory<>("quantity"));
qtyCol.setPrefWidth(60);
TableColumn<BillItem, Double> priceCol = new TableColumn<>("Unit Price");
priceCol.setCellValueFactory(new PropertyValueFactory<>("unitPrice"));
priceCol.setPrefWidth(100);
TableColumn<BillItem, Double> subCol = new TableColumn<>("Subtotal");
subCol.setCellValueFactory(new PropertyValueFactory<>("subtotal"));
subCol.setPrefWidth(100);
billItemsTable.getColumns().addAll(nameCol, qtyCol, priceCol, subCol);
// Past bills table
Label pastTitle = new Label("Past Bills");
pastTitle.setFont(Font.font("Arial", FontWeight.BOLD, 13));
TableView<Bill> pastBillsTable = new TableView<>();
ObservableList<Bill> pastBills =
FXCollections.observableArrayList(billingService.getAllBills());
pastBillsTable.setItems(pastBills);
TableColumn<Bill, String> billIdCol = new TableColumn<>("Bill ID");
billIdCol.setCellValueFactory(new PropertyValueFactory<>("billId"));
TableColumn<Bill, String> billCustCol = new TableColumn<>("Customer");
billCustCol.setCellValueFactory(new PropertyValueFactory<>("customerName"));
TableColumn<Bill, String> billDateCol = new TableColumn<>("Date");
billDateCol.setCellValueFactory(new PropertyValueFactory<>("date"));
TableColumn<Bill, Double> billAmtCol = new TableColumn<>("Amount");
billAmtCol.setCellValueFactory(new PropertyValueFactory<>("netAmount"));
pastBillsTable.getColumns().addAll(billIdCol, billCustCol, billDateCol, billAmtCol);
```

```
pastBillsTable.setPrefHeight(200);
VBox.setVgrow(billItemsTable, Priority.ALWAYS);
tableBox.getChildren().addAll(tableTitle, billItemsTable, pastTitle, pastBillsTable);
root.setCenter(tableBox);
scene = new Scene(root, 1100, 700);
}
private void updateTotals() {
double total = 0;
for (BillItem item : billItems) {
total += item.getSubtotal();
}
totalLabel.setText(String.format("Total: Rs.%.2f", total));
netLabel.setText(String.format("Net: Rs.%.2f", total));
}
public Scene getScene() {
return scene;
}
}
```

26. SupplierScreen.java

Standard CRUD screen for supplier records. Table on the right shows all suppliers, clicking a row populates the form on the left for editing. Add, Update, Delete, and Clear buttons. Search by company name filters the table in real time. Supplier IDs are auto-generated by IDGenerator.

Code for SupplierScreen.java:

```
package com.pharmacy.ui;
import com.pharmacy.model.Supplier;
import com.pharmacy.service.SupplierService;
import com.pharmacy.util.IDGenerator;
import javafx.collections.FXCollections;
import javafx.collections.ObservableList;
import javafx.geometry.Insets;
import javafx.geometry.Pos;
import javafx.scene.Scene;
import javafx.scene.control.*;
import javafx.scene.control.cell.PropertyValueFactory;
import javafx.scene.layout.*;
import javafx.scene.text.Font;
import javafx.scene.text.FontWeight;
/**
* Supplier Management Screen.
* Demonstrates: JavaFX TableView, GridPane, VBox/HBox, Event handling.
*/
public class SupplierScreen {
private Scene scene;
private PharmacyApp app;
private SupplierService supplierService;
private TableView<Supplier> tableView;
private ObservableList<Supplier> tableData;
public SupplierScreen(PharmacyApp app, SupplierService supplierService) {
this.app = app;
this.supplierService = supplierService;
createScene();
}
```

```
@SuppressWarnings("unchecked")
private void createScene() {
BorderPane root = new BorderPane();
root.setStyle("-fx-background-color: #ecf0f1;");
// Top bar
HBox topBar = new HBox(15);
topBar.setPadding(new Insets(12, 20, 12, 20));
topBar.setAlignment(Pos.CENTER_LEFT);
topBar.setStyle("-fx-background-color: #2c3e50;");
Button backBtn = new Button("< Back");
backBtn.setStyle("-fx-background-color: #7f8c8d; -fx-text-fill: white; -fx-cursor: hand;");
backBtn.setOnAction(e -> app.showDashboard(app.getUserService().getCurrentUser()));
Label titleLabel = new Label("Supplier Management");
titleLabel.setFont(Font.font("Arial", FontWeight.BOLD, 20));
titleLabel.setStyle("-fx-text-fill: white;");
topBar.getChildren().addAll(backBtn, titleLabel);
root.setTop(topBar);
// Left - Form
VBox formBox = new VBox(10);
formBox.setPadding(new Insets(15));
formBox.setPrefWidth(320);
formBox.setStyle("-fx-background-color: white; -fx-border-color: #bdc3c7; -fx-border-width: 0 1
0 0;");
Label formTitle = new Label("Add / Edit Supplier");
formTitle.setFont(Font.font("Arial", FontWeight.BOLD, 15));
GridPane grid = new GridPane();
grid.setVgap(8);
grid.setHgap(8);
TextField idField = new TextField();
idField.setEditable(false);
idField.setPromptText("Auto-generated");
TextField companyField = new TextField();
companyField.setPromptText("Company name");
TextField contactField = new TextField();
contactField.setPromptText("Contact person");
TextField phoneField = new TextField();
phoneField.setPromptText("Phone");
TextField emailField = new TextField();
emailField.setPromptText("Email");
TextField addressField = new TextField();
addressField.setPromptText("Address");
int row = 0;
grid.add(new Label("ID:"), 0, row); grid.add(idField, 1, row++);
grid.add(new Label("Company:"), 0, row); grid.add(companyField, 1, row++);
grid.add(new Label("Contact:"), 0, row); grid.add(contactField, 1, row++);
grid.add(new Label("Phone:"), 0, row); grid.add(phoneField, 1, row++);
grid.add(new Label("Email:"), 0, row); grid.add(emailField, 1, row++);
grid.add(new Label("Address:"), 0, row); grid.add(addressField, 1, row++);
Label statusLabel = new Label();
HBox btnBox = new HBox(8);
Button addBtn = new Button("Add");
addBtn.setStyle("-fx-background-color: #27ae60; -fx-text-fill: white; -fx-cursor: hand;");
Button updateBtn = new Button("Update");
updateBtn.setStyle("-fx-background-color: #2980b9; -fx-text-fill: white; -fx-cursor: hand;");
Button deleteBtn = new Button("Delete");
```

```
deleteBtn.setStyle("-fx-background-color: #e74c3c; -fx-text-fill: white; -fx-cursor: hand;");
Button clearBtn = new Button("Clear");
clearBtn.setStyle("-fx-background-color: #7f8c8d; -fx-text-fill: white; -fx-cursor: hand;");
btnBox.getChildren().addAll(addBtn, updateBtn, deleteBtn, clearBtn);
addBtn.setOnAction(e -> {
String id = IDGenerator.generateSupplierId();
Supplier sup = new Supplier(id, companyField.getText(), contactField.getText(),
phoneField.getText(), emailField.getText(), addressField.getText());
supplierService.addSupplier(sup);
refreshTable();
clearFields(idField, companyField, contactField, phoneField, emailField, addressField);
statusLabel.setText("Supplier added: " + id);
statusLabel.setStyle("-fx-text-fill: #27ae60;");
});
updateBtn.setOnAction(e -> {
String id = idField.getText();
if (id.isEmpty()) { statusLabel.setText("Select a supplier!"); return; }
Supplier sup = new Supplier(id, companyField.getText(), contactField.getText(),
phoneField.getText(), emailField.getText(), addressField.getText());
supplierService.updateSupplier(sup);
refreshTable();
statusLabel.setText("Supplier updated: " + id);
statusLabel.setStyle("-fx-text-fill: #27ae60;");
});
deleteBtn.setOnAction(e -> {
String id = idField.getText();
if (!id.isEmpty()) {
supplierService.deleteSupplier(id);
refreshTable();
clearFields(idField, companyField, contactField, phoneField, emailField, addressField);
statusLabel.setText("Supplier deleted.");
}
});
clearBtn.setOnAction(e -> {
clearFields(idField, companyField, contactField, phoneField, emailField, addressField);
statusLabel.setText("");
});
formBox.getChildren().addAll(formTitle, grid, btnBox, statusLabel);
root.setLeft(formBox);
// Center - Table
VBox tableBox = new VBox(10);
tableBox.setPadding(new Insets(15));
// Search
HBox searchBox = new HBox(10);
TextField searchField = new TextField();
searchField.setPromptText("Search by company name...");
Button searchBtn = new Button("Search");
searchBtn.setStyle("-fx-background-color: #3498db; -fx-text-fill: white; -fx-cursor: hand;");
searchBtn.setOnAction(e -> {
String q = searchField.getText().trim();
if (q.isEmpty()) refreshTable();
else {
tableData = FXCollections.observableArrayList(supplierService.searchByName(q));
tableView.setItems(tableData);
}
});
```

```
Button showAllBtn = new Button("Show All");
showAllBtn.setStyle("-fx-background-color: #7f8c8d; -fx-text-fill: white; -fx-cursor: hand;");
showAllBtn.setOnAction(e -> refreshTable());
searchBox.getChildren().addAll(searchField, searchBtn, showAllBtn);
tableView = new TableView<>();
TableColumn<Supplier, String> idCol = new TableColumn<>("ID");
idCol.setCellValueFactory(new PropertyValueFactory<>("supplierId"));
TableColumn<Supplier, String> compCol = new TableColumn<>("Company");
compCol.setCellValueFactory(new PropertyValueFactory<>("companyName"));
compCol.setPrefWidth(150);
TableColumn<Supplier, String> contCol = new TableColumn<>("Contact");
contCol.setCellValueFactory(new PropertyValueFactory<>("contactPerson"));
contCol.setPrefWidth(120);
TableColumn<Supplier, String> phoneCol = new TableColumn<>("Phone");
phoneCol.setCellValueFactory(new PropertyValueFactory<>("phone"));
phoneCol.setPrefWidth(110);
TableColumn<Supplier, String> emailCol = new TableColumn<>("Email");
emailCol.setCellValueFactory(new PropertyValueFactory<>("email"));
emailCol.setPrefWidth(150);
tableView.getColumns().addAll(idCol, compCol, contCol, phoneCol, emailCol);
refreshTable();
// Table selection listener
tableView.getSelectionModel().selectedItemProperty().addListener((obs, o, n) -> {
if (n != null) {
idField.setText(n.getSupplierId());
companyField.setText(n.getCompanyName());
contactField.setText(n.getContactPerson());
phoneField.setText(n.getPhone());
emailField.setText(n.getEmail());
addressField.setText(n.getAddress());
}
});
VBox.setVgrow(tableView, Priority.ALWAYS);
tableBox.getChildren().addAll(searchBox, tableView);
root.setCenter(tableBox);
scene = new Scene(root, 1100, 700);
}
private void refreshTable() {
tableData = FXCollections.observableArrayList(supplierService.getAllSuppliers());
tableView.setItems(tableData);
}
private void clearFields(TextField... fields) {
for (TextField f : fields) f.clear();
tableView.getSelectionModel().clearSelection();
}
public Scene getScene() { return scene; }
}
```

27. CustomerScreen.java

Same layout pattern as SupplierScreen but for customer/patient records. The search function does a case-insensitive name search through CustomerService. Customer IDs follow the CUS-xxxx format from IDGenerator. Selected table rows populate the form automatically.

Code for CustomerScreen.java:

```
package com.pharmacy.ui;
import com.pharmacy.model.Customer;
import com.pharmacy.service.CustomerService;
import com.pharmacy.util.IDGenerator;
import javafx.collections.FXCollections;
import javafx.collections.ObservableList;
import javafx.geometry.Insets;
import javafx.geometry.Pos;
import javafx.scene.Scene;
import javafx.scene.control.*;
import javafx.scene.control.cell.PropertyValueFactory;
import javafx.scene.layout.*;
import javafx.scene.text.Font;
import javafx.scene.text.FontWeight;
/**
* Customer/Patient Records Screen.
* Demonstrates: JavaFX TableView, GridPane, VBox/HBox, Event handling.
*/
public class CustomerScreen {
private Scene scene;
private PharmacyApp app;
private CustomerService customerService;
private TableView<Customer> tableView;
private ObservableList<Customer> tableData;
public CustomerScreen(PharmacyApp app, CustomerService customerService) {
this.app = app;
this.customerService = customerService;
createScene();
}
@SuppressWarnings("unchecked")
private void createScene() {
BorderPane root = new BorderPane();
root.setStyle("-fx-background-color: #ecf0f1;");
// Top bar
HBox topBar = new HBox(15);
topBar.setPadding(new Insets(12, 20, 12, 20));
topBar.setAlignment(Pos.CENTER_LEFT);
topBar.setStyle("-fx-background-color: #2c3e50;");
Button backBtn = new Button("< Back");
backBtn.setStyle("-fx-background-color: #7f8c8d; -fx-text-fill: white; -fx-cursor: hand;");
backBtn.setOnAction(e -> app.showDashboard(app.getUserService().getCurrentUser()));
Label titleLabel = new Label("Customer / Patient Records");
titleLabel.setFont(Font.font("Arial", FontWeight.BOLD, 20));
titleLabel.setStyle("-fx-text-fill: white;");
topBar.getChildren().addAll(backBtn, titleLabel);
root.setTop(topBar);
// Left - Form
VBox formBox = new VBox(10);
formBox.setPadding(new Insets(15));
formBox.setPrefWidth(320);
formBox.setStyle("-fx-background-color: white; -fx-border-color: #bdc3c7; -fx-border-width: 0 1
0 0;");
Label formTitle = new Label("Add / Edit Customer");
formTitle.setFont(Font.font("Arial", FontWeight.BOLD, 15));
GridPane grid = new GridPane();
```

```
grid.setVgap(8);
grid.setHgap(8);
TextField idField = new TextField();
idField.setEditable(false);
idField.setPromptText("Auto-generated");
TextField nameField = new TextField();
nameField.setPromptText("Full name");
TextField phoneField = new TextField();
phoneField.setPromptText("Phone number");
TextField emailField = new TextField();
emailField.setPromptText("Email");
TextField addressField = new TextField();
addressField.setPromptText("Address");
int row = 0;
grid.add(new Label("ID:"), 0, row); grid.add(idField, 1, row++);
grid.add(new Label("Name:"), 0, row); grid.add(nameField, 1, row++);
grid.add(new Label("Phone:"), 0, row); grid.add(phoneField, 1, row++);
grid.add(new Label("Email:"), 0, row); grid.add(emailField, 1, row++);
grid.add(new Label("Address:"), 0, row); grid.add(addressField, 1, row++);
Label statusLabel = new Label();
HBox btnBox = new HBox(8);
Button addBtn = new Button("Add");
addBtn.setStyle("-fx-background-color: #27ae60; -fx-text-fill: white; -fx-cursor: hand;");
Button updateBtn = new Button("Update");
updateBtn.setStyle("-fx-background-color: #2980b9; -fx-text-fill: white; -fx-cursor: hand;");
Button deleteBtn = new Button("Delete");
deleteBtn.setStyle("-fx-background-color: #e74c3c; -fx-text-fill: white; -fx-cursor: hand;");
Button clearBtn = new Button("Clear");
clearBtn.setStyle("-fx-background-color: #7f8c8d; -fx-text-fill: white; -fx-cursor: hand;");
btnBox.getChildren().addAll(addBtn, updateBtn, deleteBtn, clearBtn);
addBtn.setOnAction(e -> {
String id = IDGenerator.generateCustomerId();
Customer cust = new Customer(id, nameField.getText(), phoneField.getText(),
emailField.getText(), addressField.getText());
customerService.addCustomer(cust);
refreshTable();
clearFields(idField, nameField, phoneField, emailField, addressField);
statusLabel.setText("Customer added: " + id);
statusLabel.setStyle("-fx-text-fill: #27ae60;");
});
updateBtn.setOnAction(e -> {
String id = idField.getText();
if (id.isEmpty()) { statusLabel.setText("Select a customer!"); return; }
Customer cust = new Customer(id, nameField.getText(), phoneField.getText(),
emailField.getText(), addressField.getText());
customerService.updateCustomer(cust);
refreshTable();
statusLabel.setText("Customer updated: " + id);
statusLabel.setStyle("-fx-text-fill: #27ae60;");
});
deleteBtn.setOnAction(e -> {
String id = idField.getText();
if (!id.isEmpty()) {
customerService.deleteCustomer(id);
refreshTable();
clearFields(idField, nameField, phoneField, emailField, addressField);
```

```
statusLabel.setText("Customer deleted.");
}
});
clearBtn.setOnAction(e -> {
clearFields(idField, nameField, phoneField, emailField, addressField);
statusLabel.setText("");
});
formBox.getChildren().addAll(formTitle, grid, btnBox, statusLabel);
root.setLeft(formBox);
// Center - Table
VBox tableBox = new VBox(10);
tableBox.setPadding(new Insets(15));
HBox searchBox = new HBox(10);
TextField searchField = new TextField();
searchField.setPromptText("Search by name...");
Button searchBtn = new Button("Search");
searchBtn.setStyle("-fx-background-color: #3498db; -fx-text-fill: white; -fx-cursor: hand;");
searchBtn.setOnAction(e -> {
String q = searchField.getText().trim();
if (q.isEmpty()) refreshTable();
else {
tableData = FXCollections.observableArrayList(customerService.searchByName(q));
tableView.setItems(tableData);
}
});
Button showAllBtn = new Button("Show All");
showAllBtn.setStyle("-fx-background-color: #7f8c8d; -fx-text-fill: white; -fx-cursor: hand;");
showAllBtn.setOnAction(e -> refreshTable());
searchBox.getChildren().addAll(searchField, searchBtn, showAllBtn);
tableView = new TableView<>();
TableColumn<Customer, String> idCol = new TableColumn<>("ID");
idCol.setCellValueFactory(new PropertyValueFactory<>("customerId"));
TableColumn<Customer, String> nameCol = new TableColumn<>("Name");
nameCol.setCellValueFactory(new PropertyValueFactory<>("name"));
nameCol.setPrefWidth(150);
TableColumn<Customer, String> phoneCol = new TableColumn<>("Phone");
phoneCol.setCellValueFactory(new PropertyValueFactory<>("phone"));
phoneCol.setPrefWidth(120);
TableColumn<Customer, String> emailCol = new TableColumn<>("Email");
emailCol.setCellValueFactory(new PropertyValueFactory<>("email"));
emailCol.setPrefWidth(170);
TableColumn<Customer, String> addrCol = new TableColumn<>("Address");
addrCol.setCellValueFactory(new PropertyValueFactory<>("address"));
addrCol.setPrefWidth(180);
tableView.getColumns().addAll(idCol, nameCol, phoneCol, emailCol, addrCol);
refreshTable();
tableView.getSelectionModel().selectedItemProperty().addListener((obs, o, n) -> {
if (n != null) {
idField.setText(n.getCustomerId());
nameField.setText(n.getName());
phoneField.setText(n.getPhone());
emailField.setText(n.getEmail());
addressField.setText(n.getAddress());
}
});
```

```
VBox.setVgrow(tableView, Priority.ALWAYS);
tableBox.getChildren().addAll(searchBox, tableView);
root.setCenter(tableBox);
scene = new Scene(root, 1100, 700);
}
private void refreshTable() {
tableData = FXCollections.observableArrayList(customerService.getAllCustomers());
tableView.setItems(tableData);
}
private void clearFields(TextField... fields) {
for (TextField f : fields) f.clear();
tableView.getSelectionModel().clearSelection();
}
public Scene getScene() { return scene; }
}
```

28. PrescriptionScreen.java

Slightly more complex than the other CRUD screens because a prescription has nested items. You fill in the patient and doctor details, then add medicines one by one into a ListView showing the current list. Once you save, the full prescription (with all items) goes into PrescriptionService. Selecting a saved prescription shows its complete details in a TextArea below the main table.

Code for PrescriptionScreen.java:

```
package com.pharmacy.ui;
import com.pharmacy.model.*;
import com.pharmacy.service.*;
import com.pharmacy.util.IDGenerator;
import javafx.collections.FXCollections;
import javafx.collections.ObservableList;
import javafx.geometry.Insets;
import javafx.geometry.Pos;
import javafx.scene.Scene;
import javafx.scene.control.*;
import javafx.scene.control.cell.PropertyValueFactory;
import javafx.scene.layout.*;
import javafx.scene.text.Font;
import javafx.scene.text.FontWeight;
import java.text.SimpleDateFormat;
import java.util.Date;
/**
* Prescription Handling Screen.
* Demonstrates: JavaFX TableView, GridPane, VBox/HBox, ArrayList operations.
*/
public class PrescriptionScreen {
private Scene scene;
private PharmacyApp app;
private PrescriptionService prescriptionService;
private CustomerService customerService;
private InventoryManager inventoryManager;
private TableView<Prescription> tableView;
private ObservableList<Prescription> tableData;
public PrescriptionScreen(PharmacyApp app, PrescriptionService prescriptionService,
CustomerService customerService, InventoryManager inventoryManager) {
this.app = app;
```

```
this.prescriptionService = prescriptionService;
this.customerService = customerService;
this.inventoryManager = inventoryManager;
createScene();
}
@SuppressWarnings("unchecked")
private void createScene() {
BorderPane root = new BorderPane();
root.setStyle("-fx-background-color: #ecf0f1;");
// Top bar
HBox topBar = new HBox(15);
topBar.setPadding(new Insets(12, 20, 12, 20));
topBar.setAlignment(Pos.CENTER_LEFT);
topBar.setStyle("-fx-background-color: #2c3e50;");
Button backBtn = new Button("< Back");
backBtn.setStyle("-fx-background-color: #7f8c8d; -fx-text-fill: white; -fx-cursor: hand;");
backBtn.setOnAction(e -> app.showDashboard(app.getUserService().getCurrentUser()));
Label titleLabel = new Label("Prescription Handling");
titleLabel.setFont(Font.font("Arial", FontWeight.BOLD, 20));
titleLabel.setStyle("-fx-text-fill: white;");
topBar.getChildren().addAll(backBtn, titleLabel);
root.setTop(topBar);
// Left - Form
VBox formBox = new VBox(10);
formBox.setPadding(new Insets(15));
formBox.setPrefWidth(330);
formBox.setStyle("-fx-background-color: white; -fx-border-color: #bdc3c7; -fx-border-width: 0 1
0 0;");
Label formTitle = new Label("New Prescription");
formTitle.setFont(Font.font("Arial", FontWeight.BOLD, 15));
GridPane grid = new GridPane();
grid.setVgap(8);
grid.setHgap(8);
TextField custIdField = new TextField();
custIdField.setPromptText("Customer ID");
TextField custNameField = new TextField();
custNameField.setPromptText("Patient name");
TextField doctorField = new TextField();
doctorField.setPromptText("Doctor name");
TextField notesField = new TextField();
notesField.setPromptText("Notes/Instructions");
custIdField.textProperty().addListener((obs, o, n) -> {
if (n != null && !n.isEmpty()) {
Customer cust = customerService.getCustomerById(n.trim());
if (cust != null) custNameField.setText(cust.getName());
}
});
int r = 0;
grid.add(new Label("Customer ID:"), 0, r); grid.add(custIdField, 1, r++);
grid.add(new Label("Patient Name:"), 0, r); grid.add(custNameField, 1, r++);
grid.add(new Label("Doctor:"), 0, r); grid.add(doctorField, 1, r++);
grid.add(new Label("Notes:"), 0, r); grid.add(notesField, 1, r++);
// Prescription items
Label itemsTitle = new Label("Prescribed Medicines");
itemsTitle.setFont(Font.font("Arial", FontWeight.BOLD, 13));
```

```
TextField medIdField = new TextField();
medIdField.setPromptText("Medicine ID");
Label medNameLabel = new Label("-");
TextField medQtyField = new TextField();
medQtyField.setPromptText("Quantity");
TextField dosageField = new TextField();
dosageField.setPromptText("e.g., 1-0-1");
medIdField.textProperty().addListener((obs, o, n) -> {
if (n != null && !n.isEmpty()) {
Medicine med = inventoryManager.getMedicineById(n.trim());
if (med != null) medNameLabel.setText(med.getName());
}
});
GridPane itemGrid = new GridPane();
itemGrid.setVgap(6);
itemGrid.setHgap(6);
int ir = 0;
itemGrid.add(new Label("Med ID:"), 0, ir); itemGrid.add(medIdField, 1, ir++);
itemGrid.add(new Label("Name:"), 0, ir); itemGrid.add(medNameLabel, 1, ir++);
itemGrid.add(new Label("Qty:"), 0, ir); itemGrid.add(medQtyField, 1, ir++);
itemGrid.add(new Label("Dosage:"), 0, ir); itemGrid.add(dosageField, 1, ir++);
// Items list display
ListView<String> itemsList = new ListView<>();
itemsList.setPrefHeight(120);
java.util.ArrayList<PrescriptionItem> presItems = new java.util.ArrayList<>();
Label statusLabel = new Label();
Button addItemBtn = new Button("Add Medicine");
addItemBtn.setStyle("-fx-background-color: #3498db; -fx-text-fill: white; -fx-cursor: hand;
-fx-pref-width: 200;");
addItemBtn.setOnAction(e -> {
try {
String medId = medIdField.getText().trim();
Medicine med = inventoryManager.getMedicineById(medId);
if (med == null) { statusLabel.setText("Medicine not found!"); return; }
int qty = Integer.parseInt(medQtyField.getText().trim());
PrescriptionItem item = new PrescriptionItem(medId, med.getName(), qty, dosageField.getText());
presItems.add(item);
itemsList.getItems().add(item.toString());
medIdField.clear();
medQtyField.clear();
dosageField.clear();
medNameLabel.setText("-");
} catch (Exception ex) {
statusLabel.setText("Error: " + ex.getMessage());
}
});
Button saveBtn = new Button("Save Prescription");
saveBtn.setStyle("-fx-background-color: #27ae60; -fx-text-fill: white; -fx-font-size: 13; " +
"-fx-pref-width: 280; -fx-cursor: hand;");
saveBtn.setOnAction(e -> {
if (custNameField.getText().trim().isEmpty() || doctorField.getText().trim().isEmpty()) {
statusLabel.setText("Fill all fields!");
statusLabel.setStyle("-fx-text-fill: #e74c3c;");
return;
}
String presId = IDGenerator.generatePrescriptionId();
```

```
String date = new SimpleDateFormat("yyyy-MM-dd").format(new Date());
Prescription pres = new Prescription(presId, custIdField.getText().trim(),
custNameField.getText(), doctorField.getText(), date, notesField.getText());
for (PrescriptionItem item : presItems) {
pres.addItem(item);
}
prescriptionService.addPrescription(pres);
refreshTable();
presItems.clear();
itemsList.getItems().clear();
custIdField.clear(); custNameField.clear(); doctorField.clear(); notesField.clear();
statusLabel.setText("Prescription saved: " + presId);
statusLabel.setStyle("-fx-text-fill: #27ae60;");
});
formBox.getChildren().addAll(formTitle, grid, new Separator(),
itemsTitle, itemGrid, addItemBtn, itemsList, new Separator(),
saveBtn, statusLabel);
root.setLeft(formBox);
// Center - Table
VBox tableBox = new VBox(10);
tableBox.setPadding(new Insets(15));
Label tableTitle = new Label("All Prescriptions");
tableTitle.setFont(Font.font("Arial", FontWeight.BOLD, 15));
tableView = new TableView<>();
TableColumn<Prescription, String> idCol = new TableColumn<>("ID");
idCol.setCellValueFactory(new PropertyValueFactory<>("prescriptionId"));
TableColumn<Prescription, String> patCol = new TableColumn<>("Patient");
patCol.setCellValueFactory(new PropertyValueFactory<>("customerName"));
patCol.setPrefWidth(130);
TableColumn<Prescription, String> docCol = new TableColumn<>("Doctor");
docCol.setCellValueFactory(new PropertyValueFactory<>("doctorName"));
docCol.setPrefWidth(130);
TableColumn<Prescription, String> dateCol = new TableColumn<>("Date");
dateCol.setCellValueFactory(new PropertyValueFactory<>("date"));
TableColumn<Prescription, String> notesCol = new TableColumn<>("Notes");
notesCol.setCellValueFactory(new PropertyValueFactory<>("notes"));
notesCol.setPrefWidth(180);
tableView.getColumns().addAll(idCol, patCol, docCol, dateCol, notesCol);
refreshTable();
// Details area
TextArea detailsArea = new TextArea();
detailsArea.setEditable(false);
detailsArea.setPrefHeight(150);
detailsArea.setPromptText("Select a prescription to view details...");
tableView.getSelectionModel().selectedItemProperty().addListener((obs, o, n) -> {
if (n != null) {
StringBuilder sb = new StringBuilder();
sb.append("Prescription: ").append(n.getPrescriptionId()).append("\n");
sb.append("Patient: ").append(n.getCustomerName()).append("\n");
sb.append("Doctor: Dr. ").append(n.getDoctorName()).append("\n");
sb.append("Date: ").append(n.getDate()).append("\n");
sb.append("Notes: ").append(n.getNotes()).append("\n\n");
sb.append("Prescribed Medicines:\n");
for (PrescriptionItem item : n.getItems()) {
sb.append(" - ").append(item.toString()).append("\n");
```

```
}
detailsArea.setText(sb.toString());
}
});
Button deleteBtn = new Button("Delete Selected");
deleteBtn.setStyle("-fx-background-color: #e74c3c; -fx-text-fill: white; -fx-cursor: hand;");
deleteBtn.setOnAction(e -> {
Prescription sel = tableView.getSelectionModel().getSelectedItem();
if (sel != null) {
prescriptionService.deletePrescription(sel.getPrescriptionId());
refreshTable();
detailsArea.clear();
}
});
VBox.setVgrow(tableView, Priority.ALWAYS);
tableBox.getChildren().addAll(tableTitle, tableView, new Label("Details:"), detailsArea,
deleteBtn);
root.setCenter(tableBox);
scene = new Scene(root, 1100, 700);
}
private void refreshTable() {
tableData = FXCollections.observableArrayList(prescriptionService.getAllPrescriptions());
tableView.setItems(tableData);
}
public Scene getScene() { return scene; }
}
```

29. ReportsScreen.java

The reports screen has a menu on the left side with buttons for different report types. The default view is a summary dashboard showing coloured cards with counts for medicines, customers, bills, and total sales. Other views show the full sales report, stock levels, expiry alerts (pulled from the running ExpiryAlertThread), and low stock items. The backup button runs BackupService in a new thread and shows a progress indicator while it runs.

Code for ReportsScreen.java:

```
package com.pharmacy.ui;
import com.pharmacy.model.Bill;
import com.pharmacy.model.Medicine;
import com.pharmacy.service.*;
import javafx.application.Platform;
import javafx.collections.FXCollections;
import javafx.collections.ObservableList;
import javafx.geometry.Insets;
import javafx.geometry.Pos;
import javafx.scene.Scene;
import javafx.scene.control.*;
import javafx.scene.control.cell.PropertyValueFactory;
import javafx.scene.layout.*;
import javafx.scene.text.Font;
import javafx.scene.text.FontWeight;
import java.text.SimpleDateFormat;
import java.util.ArrayList;
import java.util.Calendar;
import java.util.Date;
```

```
/**
* Reports & Alerts Screen - Sales reports, stock alerts, expiry alerts, backups.
* Demonstrates: JavaFX TableView, VBox/HBox, Multithreading integration, Backup service.
*/
public class ReportsScreen {
private Scene scene;
private PharmacyApp app;
private InventoryManager inventoryManager;
private BillingService billingService;
private CustomerService customerService;
private SupplierService supplierService;
private ExpiryAlertThread expiryAlertThread;
private StockAlertThread stockAlertRunnable;
public ReportsScreen(PharmacyApp app, InventoryManager inventoryManager,
BillingService billingService, CustomerService customerService,
SupplierService supplierService,
ExpiryAlertThread expiryAlertThread, StockAlertThread stockAlertRunnable) {
this.app = app;
this.inventoryManager = inventoryManager;
this.billingService = billingService;
this.customerService = customerService;
this.supplierService = supplierService;
this.expiryAlertThread = expiryAlertThread;
this.stockAlertRunnable = stockAlertRunnable;
createScene();
}
@SuppressWarnings("unchecked")
private void createScene() {
BorderPane root = new BorderPane();
root.setStyle("-fx-background-color: #ecf0f1;");
// Top bar
HBox topBar = new HBox(15);
topBar.setPadding(new Insets(12, 20, 12, 20));
topBar.setAlignment(Pos.CENTER_LEFT);
topBar.setStyle("-fx-background-color: #2c3e50;");
Button backBtn = new Button("< Back");
backBtn.setStyle("-fx-background-color: #7f8c8d; -fx-text-fill: white; -fx-cursor: hand;");
backBtn.setOnAction(e -> app.showDashboard(app.getUserService().getCurrentUser()));
Label titleLabel = new Label("Reports & Alerts");
titleLabel.setFont(Font.font("Arial", FontWeight.BOLD, 20));
titleLabel.setStyle("-fx-text-fill: white;");
topBar.getChildren().addAll(backBtn, titleLabel);
root.setTop(topBar);
// Left - Report categories
VBox leftMenu = new VBox(8);
leftMenu.setPadding(new Insets(15));
leftMenu.setPrefWidth(200);
leftMenu.setStyle("-fx-background-color: white; -fx-border-color: #bdc3c7; -fx-border-width: 0
1 0 0;");
Label menuTitle = new Label("Report Types");
menuTitle.setFont(Font.font("Arial", FontWeight.BOLD, 14));
// Content area in center
VBox contentArea = new VBox(10);
contentArea.setPadding(new Insets(15));
Button salesReportBtn = createMenuButton("Sales Report", "#27ae60");
```

```
Button stockReportBtn = createMenuButton("Stock Report", "#3498db");
Button expiryAlertBtn = createMenuButton("Expiry Alerts", "#e74c3c");
Button lowStockBtn = createMenuButton("Low Stock Alert", "#e67e22");
Button summaryBtn = createMenuButton("Summary", "#9b59b6");
Button backupBtn = createMenuButton("Backup Data", "#34495e");
// Sales Report
salesReportBtn.setOnAction(e -> showSalesReport(contentArea));
// Stock Report
stockReportBtn.setOnAction(e -> showStockReport(contentArea));
// Expiry Alerts (from alert thread)
expiryAlertBtn.setOnAction(e -> showExpiryAlerts(contentArea));
// Low Stock Alert (from stock thread)
lowStockBtn.setOnAction(e -> showLowStockAlerts(contentArea));
// Summary
summaryBtn.setOnAction(e -> showSummary(contentArea));
// Backup
backupBtn.setOnAction(e -> runBackup(contentArea));
leftMenu.getChildren().addAll(menuTitle, new Separator(),
salesReportBtn, stockReportBtn, expiryAlertBtn,
lowStockBtn, summaryBtn, new Separator(), backupBtn);
root.setLeft(leftMenu);
// Initial content
showSummary(contentArea);
root.setCenter(contentArea);
scene = new Scene(root, 1100, 700);
}
private void showSalesReport(VBox contentArea) {
contentArea.getChildren().clear();
Label title = new Label("Sales Report");
title.setFont(Font.font("Arial", FontWeight.BOLD, 18));
String today = new SimpleDateFormat("yyyy-MM-dd").format(new Date());
Label totalSales = new Label(String.format("Total Sales (All Time): Rs.%.2f",
billingService.getTotalSales()));
totalSales.setFont(Font.font("Arial", FontWeight.BOLD, 14));
totalSales.setStyle("-fx-text-fill: #27ae60;");
Label todaySales = new Label(String.format("Today's Sales: Rs.%.2f",
billingService.getTotalSalesByDate(today)));
todaySales.setFont(Font.font("Arial", FontWeight.BOLD, 14));
Label billCount = new Label("Total Bills: " + billingService.getBillCount());
// Bills table
TableView<Bill> billTable = new TableView<>();
ObservableList<Bill> bills = FXCollections.observableArrayList(billingService.getAllBills());
billTable.setItems(bills);
TableColumn<Bill, String> idCol = new TableColumn<>("Bill ID");
idCol.setCellValueFactory(new PropertyValueFactory<>("billId"));
TableColumn<Bill, String> custCol = new TableColumn<>("Customer");
custCol.setCellValueFactory(new PropertyValueFactory<>("customerName"));
custCol.setPrefWidth(150);
TableColumn<Bill, String> dateCol = new TableColumn<>("Date");
dateCol.setCellValueFactory(new PropertyValueFactory<>("date"));
TableColumn<Bill, Double> totalCol = new TableColumn<>("Total");
totalCol.setCellValueFactory(new PropertyValueFactory<>("totalAmount"));
TableColumn<Bill, Double> discCol = new TableColumn<>("Discount");
discCol.setCellValueFactory(new PropertyValueFactory<>("discount"));
TableColumn<Bill, Double> netCol = new TableColumn<>("Net Amount");
```

```
netCol.setCellValueFactory(new PropertyValueFactory<>("netAmount"));
netCol.setPrefWidth(100);
TableColumn<Bill, String> byCol = new TableColumn<>("Billed By");
byCol.setCellValueFactory(new PropertyValueFactory<>("billedBy"));
billTable.getColumns().addAll(idCol, custCol, dateCol, totalCol, discCol, netCol, byCol);
VBox.setVgrow(billTable, Priority.ALWAYS);
contentArea.getChildren().addAll(title, totalSales, todaySales, billCount, billTable);
}
@SuppressWarnings("unchecked")
private void showStockReport(VBox contentArea) {
contentArea.getChildren().clear();
Label title = new Label("Stock Report");
title.setFont(Font.font("Arial", FontWeight.BOLD, 18));
Label countLabel = new Label("Total Medicines in Inventory: " +
inventoryManager.getMedicineCount());
countLabel.setFont(Font.font("Arial", FontWeight.BOLD, 14));
TableView<Medicine> table = new TableView<>();
ObservableList<Medicine> data =
FXCollections.observableArrayList(inventoryManager.getSortedByName());
table.setItems(data);
TableColumn<Medicine, String> idCol = new TableColumn<>("ID");
idCol.setCellValueFactory(new PropertyValueFactory<>("medicineId"));
TableColumn<Medicine, String> nameCol = new TableColumn<>("Name");
nameCol.setCellValueFactory(new PropertyValueFactory<>("name"));
nameCol.setPrefWidth(170);
TableColumn<Medicine, String> catCol = new TableColumn<>("Category");
catCol.setCellValueFactory(new PropertyValueFactory<>("category"));
TableColumn<Medicine, Integer> qtyCol = new TableColumn<>("Quantity");
qtyCol.setCellValueFactory(new PropertyValueFactory<>("quantity"));
TableColumn<Medicine, Double> priceCol = new TableColumn<>("Price");
priceCol.setCellValueFactory(new PropertyValueFactory<>("price"));
TableColumn<Medicine, String> expCol = new TableColumn<>("Expiry");
expCol.setCellValueFactory(new PropertyValueFactory<>("expiryDate"));
expCol.setPrefWidth(100);
table.getColumns().addAll(idCol, nameCol, catCol, qtyCol, priceCol, expCol);
VBox.setVgrow(table, Priority.ALWAYS);
contentArea.getChildren().addAll(title, countLabel, table);
}
@SuppressWarnings("unchecked")
private void showExpiryAlerts(VBox contentArea) {
contentArea.getChildren().clear();
Label title = new Label("Expiry Alerts");
title.setFont(Font.font("Arial", FontWeight.BOLD, 18));
title.setStyle("-fx-text-fill: #e74c3c;");
SimpleDateFormat sdf = new SimpleDateFormat("yyyy-MM-dd");
String currentDate = sdf.format(new Date());
Calendar cal = Calendar.getInstance();
cal.add(Calendar.DAY_OF_MONTH, 30);
String thresholdDate = sdf.format(cal.getTime());
// Expired medicines
ArrayList<Medicine> expired = inventoryManager.getExpiredMedicines(currentDate);
Label expiredTitle = new Label("EXPIRED Medicines (" + expired.size() + ")");
expiredTitle.setFont(Font.font("Arial", FontWeight.BOLD, 14));
expiredTitle.setStyle("-fx-text-fill: #e74c3c;");
TableView<Medicine> expiredTable = new TableView<>();
```

```
expiredTable.setItems(FXCollections.observableArrayList(expired));
addMedicineColumns(expiredTable);
expiredTable.setPrefHeight(200);
// Expiring soon
ArrayList<Medicine> expiringSoon = inventoryManager.getExpiringSoon(currentDate,
thresholdDate);
Label expiringTitle = new Label("Expiring Within 30 Days (" + expiringSoon.size() + ")");
expiringTitle.setFont(Font.font("Arial", FontWeight.BOLD, 14));
expiringTitle.setStyle("-fx-text-fill: #e67e22;");
TableView<Medicine> expiringTable = new TableView<>();
expiringTable.setItems(FXCollections.observableArrayList(expiringSoon));
addMedicineColumns(expiringTable);
VBox.setVgrow(expiringTable, Priority.ALWAYS);
contentArea.getChildren().addAll(title, expiredTitle, expiredTable, expiringTitle,
expiringTable);
}
@SuppressWarnings("unchecked")
private void showLowStockAlerts(VBox contentArea) {
contentArea.getChildren().clear();
Label title = new Label("Low Stock Alerts (< 10 units)");
title.setFont(Font.font("Arial", FontWeight.BOLD, 18));
title.setStyle("-fx-text-fill: #e67e22;");
ArrayList<Medicine> lowStock = inventoryManager.getLowStockMedicines();
Label countLabel = new Label("Low Stock Items: " + lowStock.size());
countLabel.setFont(Font.font("Arial", FontWeight.BOLD, 14));
TableView<Medicine> table = new TableView<>();
table.setItems(FXCollections.observableArrayList(lowStock));
addMedicineColumns(table);
VBox.setVgrow(table, Priority.ALWAYS);
contentArea.getChildren().addAll(title, countLabel, table);
}
private void showSummary(VBox contentArea) {
contentArea.getChildren().clear();
Label title = new Label("System Summary");
title.setFont(Font.font("Arial", FontWeight.BOLD, 18));
GridPane summaryGrid = new GridPane();
summaryGrid.setVgap(15);
summaryGrid.setHgap(20);
summaryGrid.setPadding(new Insets(20));
addSummaryCard(summaryGrid, 0, 0, "Total Medicines",
String.valueOf(inventoryManager.getMedicineCount()), "#3498db");
addSummaryCard(summaryGrid, 1, 0, "Total Customers",
String.valueOf(customerService.getCustomerCount()), "#9b59b6");
addSummaryCard(summaryGrid, 2, 0, "Total Suppliers",
String.valueOf(supplierService.getSupplierCount()), "#e67e22");
addSummaryCard(summaryGrid, 0, 1, "Total Bills",
String.valueOf(billingService.getBillCount()), "#27ae60");
addSummaryCard(summaryGrid, 1, 1, "Total Sales",
String.format("Rs.%.2f", billingService.getTotalSales()), "#2c3e50");
String currentDate = new SimpleDateFormat("yyyy-MM-dd").format(new Date());
addSummaryCard(summaryGrid, 2, 1, "Low Stock Items",
String.valueOf(inventoryManager.getLowStockMedicines().size()), "#e74c3c");
// Thread status
Label threadStatus = new Label("Background Threads:");
threadStatus.setFont(Font.font("Arial", FontWeight.BOLD, 14));
```

```
Label expiryThreadLabel = new Label("Expiry Alert Thread: " +
(expiryAlertThread != null && expiryAlertThread.isAlive() ? "RUNNING" : "STOPPED"));
expiryThreadLabel.setStyle(expiryAlertThread != null && expiryAlertThread.isAlive() ?
"-fx-text-fill: #27ae60;" : "-fx-text-fill: #e74c3c;");
Label stockThreadLabel = new Label("Stock Alert Thread: RUNNING (daemon)");
stockThreadLabel.setStyle("-fx-text-fill: #27ae60;");
contentArea.getChildren().addAll(title, summaryGrid, new Separator(),
threadStatus, expiryThreadLabel, stockThreadLabel);
}
private void addSummaryCard(GridPane grid, int col, int row, String label, String value, String
color) {
VBox card = new VBox(5);
card.setPadding(new Insets(15));
card.setPrefSize(180, 80);
card.setAlignment(Pos.CENTER);
card.setStyle("-fx-background-color: " + color + "; -fx-background-radius: 8;");
Label valLabel = new Label(value);
valLabel.setFont(Font.font("Arial", FontWeight.BOLD, 20));
valLabel.setStyle("-fx-text-fill: white;");
Label nameLabel = new Label(label);
nameLabel.setStyle("-fx-text-fill: #ecf0f1; -fx-font-size: 12;");
card.getChildren().addAll(valLabel, nameLabel);
grid.add(card, col, row);
}
private void runBackup(VBox contentArea) {
contentArea.getChildren().clear();
Label title = new Label("Data Backup");
title.setFont(Font.font("Arial", FontWeight.BOLD, 18));
Label statusLabel = new Label("Starting backup...");
statusLabel.setStyle("-fx-text-fill: #3498db; -fx-font-size: 14;");
ProgressIndicator progress = new ProgressIndicator();
progress.setPrefSize(50, 50);
contentArea.getChildren().addAll(title, progress, statusLabel);
// Run backup in a separate thread (BackupService implements Runnable)
BackupService backupService = new BackupService(
inventoryManager, customerService, supplierService,
billingService, app.getPrescriptionService());
backupService.setCallback(message -> {
Platform.runLater(() -> {
statusLabel.setText(message);
statusLabel.setStyle("-fx-text-fill: #27ae60; -fx-font-size: 14;");
progress.setProgress(1.0);
Label pathLabel = new Label("Backup files saved in: data/ directory");
pathLabel.setStyle("-fx-font-size: 12;");
Label filesLabel = new Label(
"Files created:\n" +
" - backup_medicines.txt\n" +
" - backup_customers.txt\n" +
" - backup_suppliers.txt\n" +
" - backup_bills.txt\n" +
" - backup_prescriptions.txt");
filesLabel.setStyle("-fx-font-size: 12;");
contentArea.getChildren().addAll(pathLabel, filesLabel);
});
});
```

```
Thread backupThread = new Thread(backupService, "BackupThread");
backupThread.setDaemon(true);
backupThread.start();
}
@SuppressWarnings("unchecked")
private void addMedicineColumns(TableView<Medicine> table) {
TableColumn<Medicine, String> idCol = new TableColumn<>("ID");
idCol.setCellValueFactory(new PropertyValueFactory<>("medicineId"));
TableColumn<Medicine, String> nameCol = new TableColumn<>("Name");
nameCol.setCellValueFactory(new PropertyValueFactory<>("name"));
nameCol.setPrefWidth(150);
TableColumn<Medicine, String> catCol = new TableColumn<>("Category");
catCol.setCellValueFactory(new PropertyValueFactory<>("category"));
TableColumn<Medicine, Integer> qtyCol = new TableColumn<>("Qty");
qtyCol.setCellValueFactory(new PropertyValueFactory<>("quantity"));
TableColumn<Medicine, Double> priceCol = new TableColumn<>("Price");
priceCol.setCellValueFactory(new PropertyValueFactory<>("price"));
TableColumn<Medicine, String> expCol = new TableColumn<>("Expiry");
expCol.setCellValueFactory(new PropertyValueFactory<>("expiryDate"));
expCol.setPrefWidth(100);
table.getColumns().addAll(idCol, nameCol, catCol, qtyCol, priceCol, expCol);
}
private Button createMenuButton(String text, String color) {
Button btn = new Button(text);
btn.setPrefWidth(170);
btn.setStyle("-fx-background-color: " + color + "; -fx-text-fill: white; " +
"-fx-font-size: 12; -fx-cursor: hand; -fx-alignment: center-left;");
return btn;
}
public Scene getScene() { return scene; }
}
```

Util Package (com.pharmacy.util)

30. DataStore.java

This is the generic persistence class that all five service classes use. It's declared as DataStore<T extends Serializable>, so it works with any model class that implements Serializable. saveAll() takes an ArrayList and writes it to a .dat file using ObjectOutputStream wrapped around FileOutputStream. loadAll() reads it back with ObjectInputStream and FileInputStream. backupToText() writes a readable version using FileWriter. All methods are synchronized to handle concurrent access from background threads.

Code for DataStore.java:

```
package com.pharmacy.util;
import java.io.*;
import java.util.ArrayList;
/**
* Generic data store for serialization and deserialization.
* Demonstrates: Generics, Serialization/Deserialization, File I/O.
*
* @param <T> The type of objects to store (must be Serializable)
*/
public class DataStore<T extends Serializable> {
private String filePath;
```

```
public DataStore(String filePath) {
this.filePath = filePath;
}
/**
* Save a list of objects using Serialization (ObjectOutputStream).
* Demonstrates: FileOutputStream, ObjectOutputStream, Serialization.
*/
public synchronized void saveAll(ArrayList<T> items) {
try (FileOutputStream fos = new FileOutputStream(filePath);
ObjectOutputStream oos = new ObjectOutputStream(fos)) {
oos.writeObject(items);
} catch (IOException e) {
System.err.println("Error saving data to " + filePath + ": " + e.getMessage());
}
}
/**
* Load a list of objects using Deserialization (ObjectInputStream).
* Demonstrates: FileInputStream, ObjectInputStream, Deserialization.
*/
@SuppressWarnings("unchecked")
public synchronized ArrayList<T> loadAll() {
File file = new File(filePath);
if (!file.exists()) {
return new ArrayList<>();
}
try (FileInputStream fis = new FileInputStream(filePath);
ObjectInputStream ois = new ObjectInputStream(fis)) {
return (ArrayList<T>) ois.readObject();
} catch (IOException | ClassNotFoundException e) {
System.err.println("Error loading data from " + filePath + ": " + e.getMessage());
return new ArrayList<>();
}
}
/**
* Backup data to a text file using FileWriter.
* Demonstrates: FileWriter usage.
*/
public void backupToText(ArrayList<T> items, String backupPath) {
try (FileWriter writer = new FileWriter(backupPath)) {
writer.write("=== DATA BACKUP ===\n");
writer.write("Source: " + filePath + "\n");
writer.write("Records: " + items.size() + "\n");
writer.write("====================\n\n");
for (T item : items) {
writer.write(item.toString() + "\n");
}
writer.write("\n=== END OF BACKUP ===\n");
} catch (IOException e) {
System.err.println("Error creating backup: " + e.getMessage());
}
}
/**
* Read backup file content using FileReader.
* Demonstrates: FileReader usage.
*/
public String readBackup(String backupPath) {
```

```
StringBuilder content = new StringBuilder();
File file = new File(backupPath);
if (!file.exists()) {
return "No backup found.";
}
try (FileReader reader = new FileReader(backupPath)) {
int ch;
while ((ch = reader.read()) != -1) {
content.append((char) ch);
}
} catch (IOException e) {
System.err.println("Error reading backup: " + e.getMessage());
}
return content.toString();
}
public String getFilePath() {
return filePath;
}
}
```

31. IDGenerator.java

A small but important utility class. It keeps static integer counters for each type of record and generates IDs like MED1001, CUS2001, SUP3001 etc. Every generation method is synchronized so that even if two threads try to get an ID at the same time, they'll always get different values. There's also an updateCounters() method that gets called on startup to continue from where the last saved data left off.

Code for IDGenerator.java:

```
package com.pharmacy.util;
/**
* Utility class for generating unique IDs.
* Demonstrates: Static methods, synchronized for thread safety.
*/
public class IDGenerator {
private static int medicineCounter = 1000;
private static int customerCounter = 2000;
private static int supplierCounter = 3000;
private static int billCounter = 4000;
private static int prescriptionCounter = 5000;
// Synchronized to ensure thread-safe ID generation
public static synchronized String generateMedicineId() {
return "MED" + (++medicineCounter);
}
public static synchronized String generateCustomerId() {
return "CUS" + (++customerCounter);
}
public static synchronized String generateSupplierId() {
return "SUP" + (++supplierCounter);
}
public static synchronized String generateBillId() {
return "BIL" + (++billCounter);
}
public static synchronized String generatePrescriptionId() {
return "PRE" + (++prescriptionCounter);
```

```
}
// Update counters based on existing data
public static void updateCounters(int medCount, int cusCount, int supCount,
int bilCount, int preCount) {
if (medCount > medicineCounter) medicineCounter = medCount;
if (cusCount > customerCounter) customerCounter = cusCount;
if (supCount > supplierCounter) supplierCounter = supCount;
if (bilCount > billCounter) billCounter = bilCount;
if (preCount > prescriptionCounter) prescriptionCounter = preCount;
}
}
```

32. SampleDataLoader.java

On the very first run, before any real data exists, this class populates the system with 10 sample medicines (covering all the enum categories), 3 sample customers, and 3 sample suppliers. It checks whether medicines already exist before loading anything so it never doubles up on runs after the first.

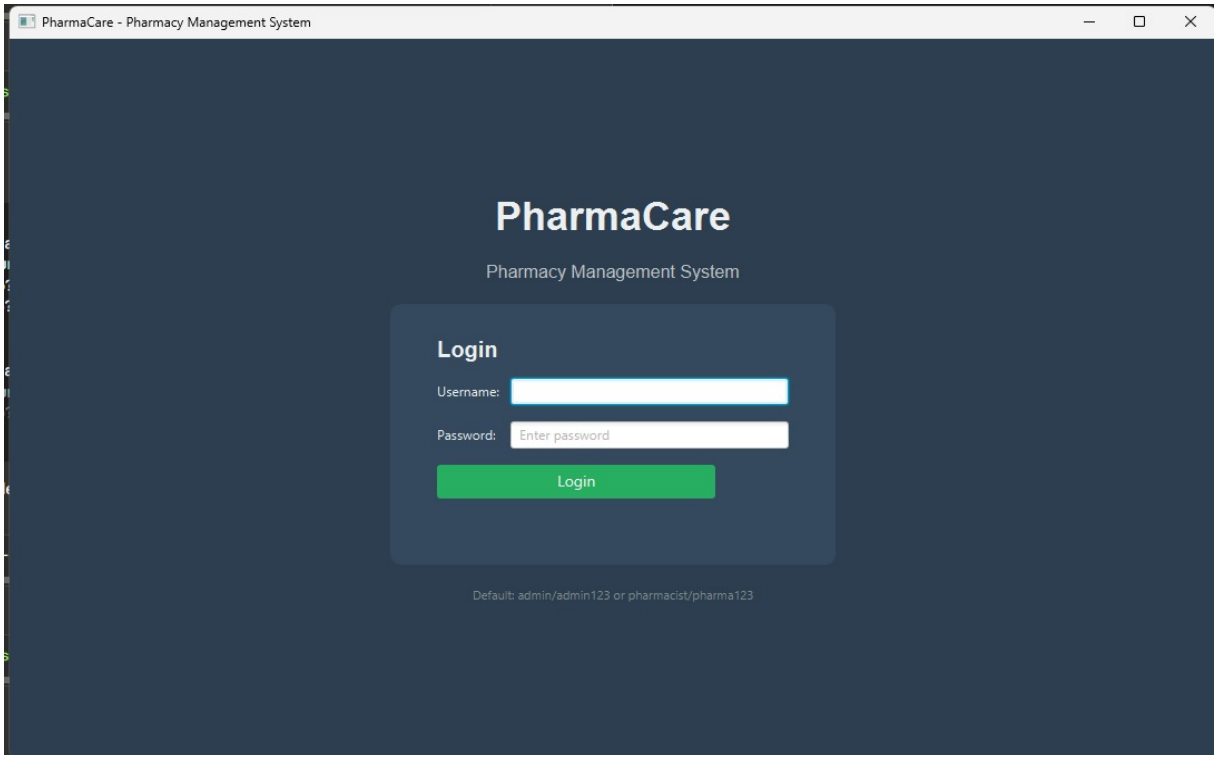
Code for SampleDataLoader.java:

```
package com.pharmacy.util;
import com.pharmacy.model.*;
import com.pharmacy.service.*;
/**
* Loads sample data for demonstration purposes.
* Demonstrates: Object creation, Enum usage, ArrayList operations.
*/
public class SampleDataLoader {
public static void loadSampleData(InventoryManager inventoryManager,
CustomerService customerService,
SupplierService supplierService) {
// Only load if no data exists
if (inventoryManager.getMedicineCount() > 0) {
return;
}
// Sample Suppliers
Supplier s1 = new Supplier("SUP3001", "PharmaCorp India", "Rajesh Kumar",
"9876543210", "rajesh@pharmacorp.com", "Mumbai, Maharashtra");
Supplier s2 = new Supplier("SUP3002", "MediLife Supplies", "Priya Sharma",
"9876543211", "priya@medilife.com", "Delhi, India");
Supplier s3 = new Supplier("SUP3003", "HealthFirst Pharma", "Amit Patel",
"9876543212", "amit@healthfirst.com", "Ahmedabad, Gujarat");
supplierService.addSupplier(s1);
supplierService.addSupplier(s2);
supplierService.addSupplier(s3);
// Sample Medicines
inventoryManager.addMedicine(new Medicine("MED1001", "Paracetamol 500mg", "Sun Pharma",
MedicineCategory.TABLET, "BT001", "2026-12-15", 500, 12.50, "SUP3001"));
inventoryManager.addMedicine(new Medicine("MED1002", "Amoxicillin 250mg", "Cipla",
MedicineCategory.CAPSULE, "BT002", "2026-06-20", 200, 45.00, "SUP3001"));
inventoryManager.addMedicine(new Medicine("MED1003", "Cough Syrup", "Himalaya",
MedicineCategory.SYRUP, "BT003", "2026-03-10", 150, 85.00, "SUP3002"));
inventoryManager.addMedicine(new Medicine("MED1004", "Insulin Injection", "Novo Nordisk",
MedicineCategory.INJECTION, "BT004", "2025-08-25", 50, 350.00, "SUP3003"));
inventoryManager.addMedicine(new Medicine("MED1005", "Dettol Ointment", "Reckitt",
```

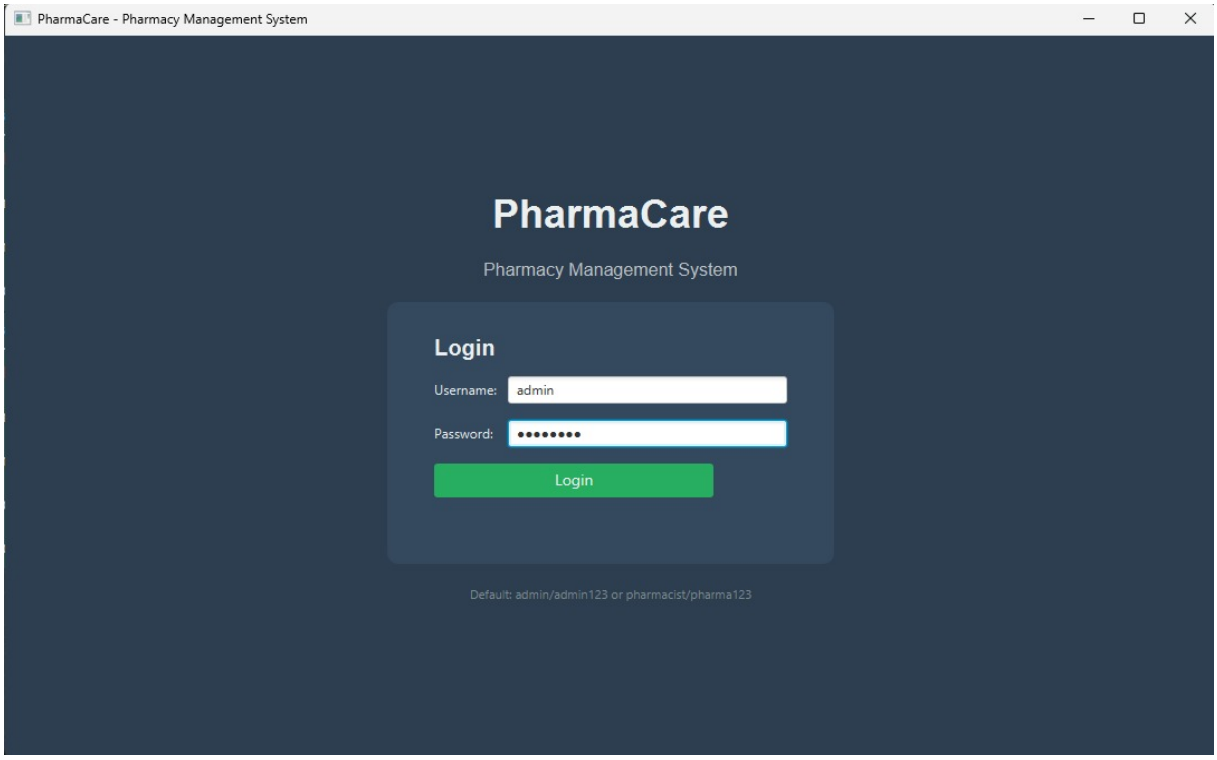
```
MedicineCategory.OINTMENT, "BT005", "2027-01-30", 300, 65.00, "SUP3002"));
inventoryManager.addMedicine(new Medicine("MED1006", "Eye Drops", "Alcon",
MedicineCategory.DROPS, "BT006", "2026-09-15", 8, 120.00, "SUP3003"));
inventoryManager.addMedicine(new Medicine("MED1007", "Salbutamol Inhaler", "Cipla",
MedicineCategory.INHALER, "BT007", "2026-11-20", 5, 250.00, "SUP3001"));
inventoryManager.addMedicine(new Medicine("MED1008", "ORS Powder", "WHO Standard",
MedicineCategory.POWDER, "BT008", "2027-06-10", 1000, 15.00, "SUP3002"));
inventoryManager.addMedicine(new Medicine("MED1009", "Azithromycin 500mg", "Zydus",
MedicineCategory.TABLET, "BT009", "2026-04-05", 100, 75.00, "SUP3001"));
inventoryManager.addMedicine(new Medicine("MED1010", "Vitamin D3", "HealthVit",
MedicineCategory.CAPSULE, "BT010", "2026-10-30", 400, 30.00, "SUP3003"));
// Sample Customers
customerService.addCustomer(new Customer("CUS2001", "Rahul Verma", "9123456789",
"rahul@email.com", "Bangalore, Karnataka"));
customerService.addCustomer(new Customer("CUS2002", "Sneha Iyer", "9123456790",
"sneha@email.com", "Chennai, Tamil Nadu"));
customerService.addCustomer(new Customer("CUS2003", "Mohammed Ali", "9123456791",
"ali@email.com", "Hyderabad, Telangana"));
// Update ID counters
IDGenerator.updateCounters(1010, 2003, 3003, 4000, 5000);
}
}
```

Results

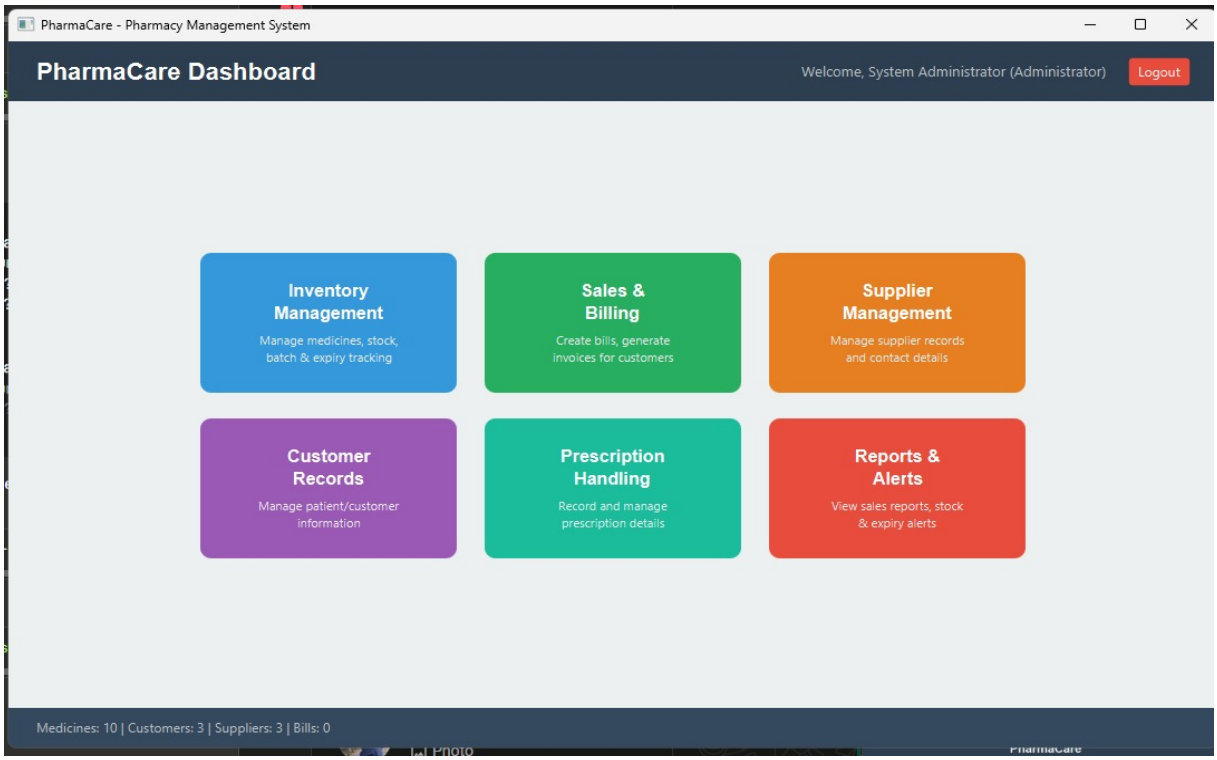
Login Screen



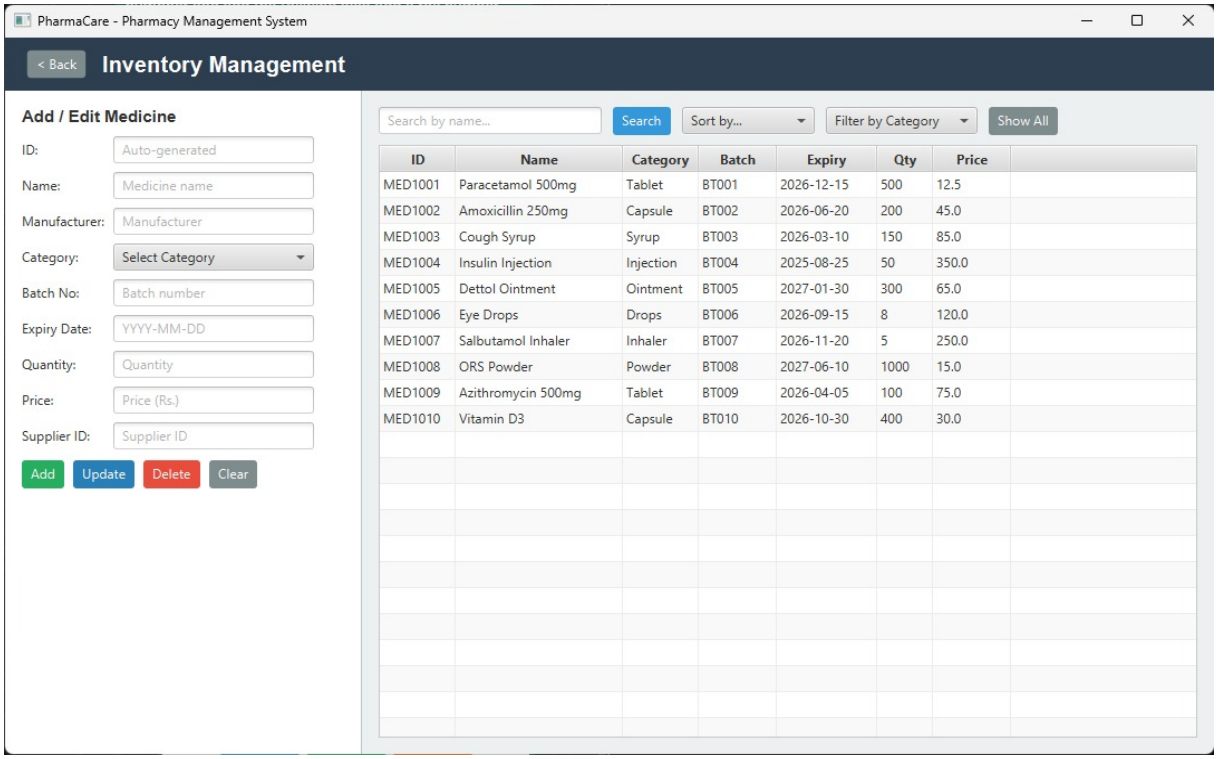
Login Screen - with Credentials



Dashboard Screen

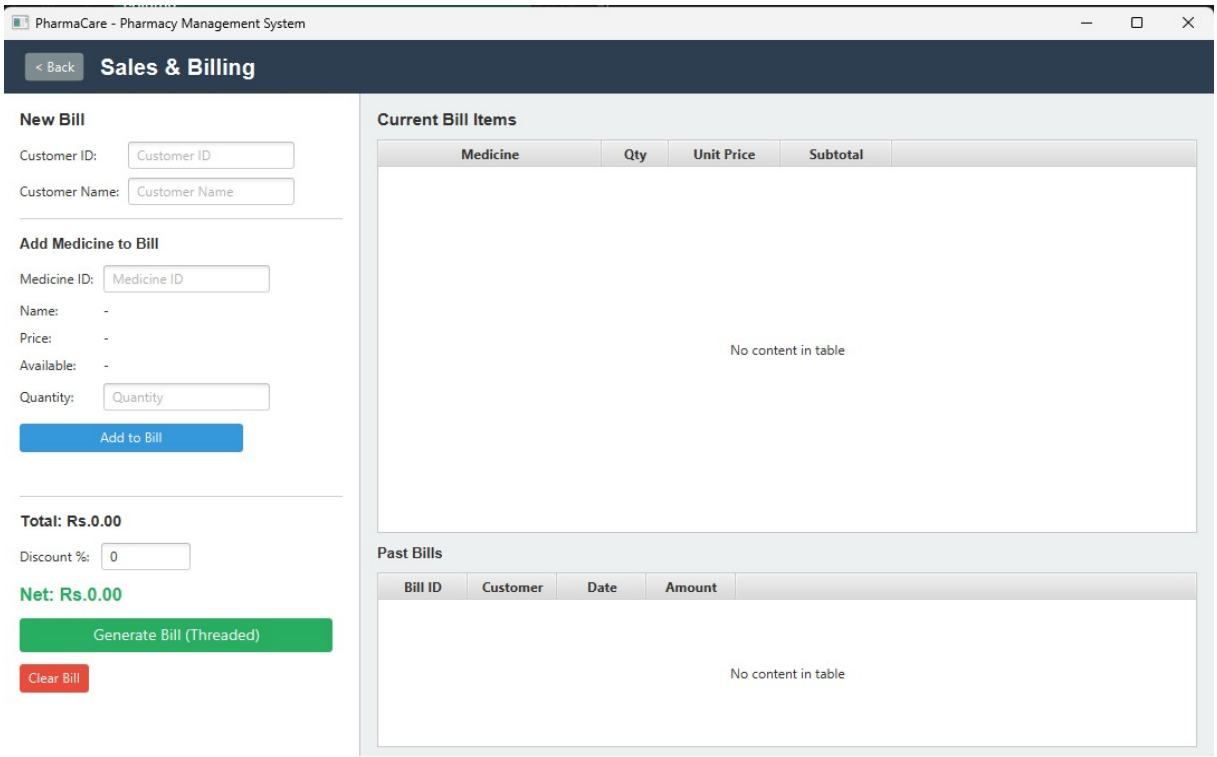


Inventory Management Screen

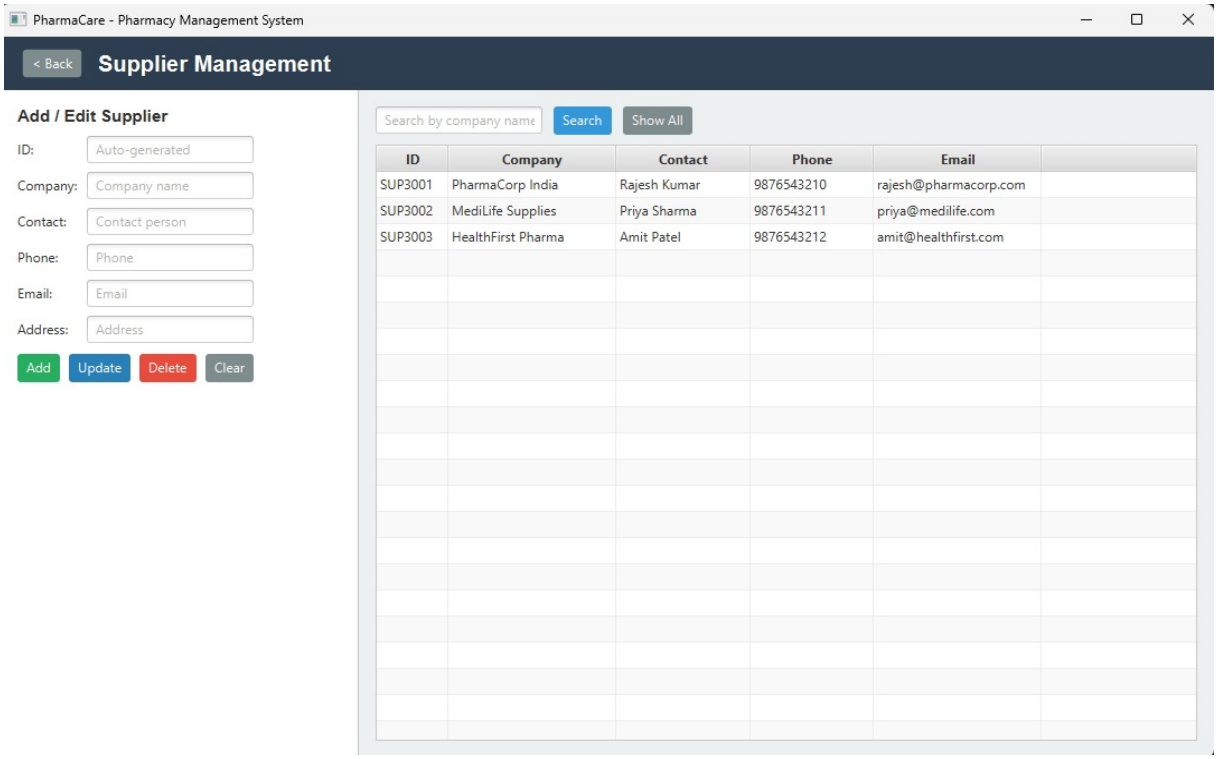


ID	Name	Category	Batch	Expiry	Qty	Price
MED1001	Paracetamol 500mg	Tablet	BT001	2026-12-15	500	12.5
MED1002	Amoxicillin 250mg	Capsule	BT002	2026-06-20	200	45.0
MED1003	Cough Syrup	Syrup	BT003	2026-03-10	150	85.0
MED1004	Insulin Injection	Injection	BT004	2025-08-25	50	350.0
MED1005	Dettol Ointment	Ointment	BT005	2027-01-30	300	65.0
MED1006	Eye Drops	Drops	BT006	2026-09-15	8	120.0
MED1007	Salbutamol Inhaler	Inhaler	BT007	2026-11-20	5	250.0
MED1008	ORS Powder	Powder	BT008	2027-06-10	1000	15.0
MED1009	Azithromycin 500mg	Tablet	BT009	2026-04-05	100	75.0
MED1010	Vitamin D3	Capsule	BT010	2026-10-30	400	30.0

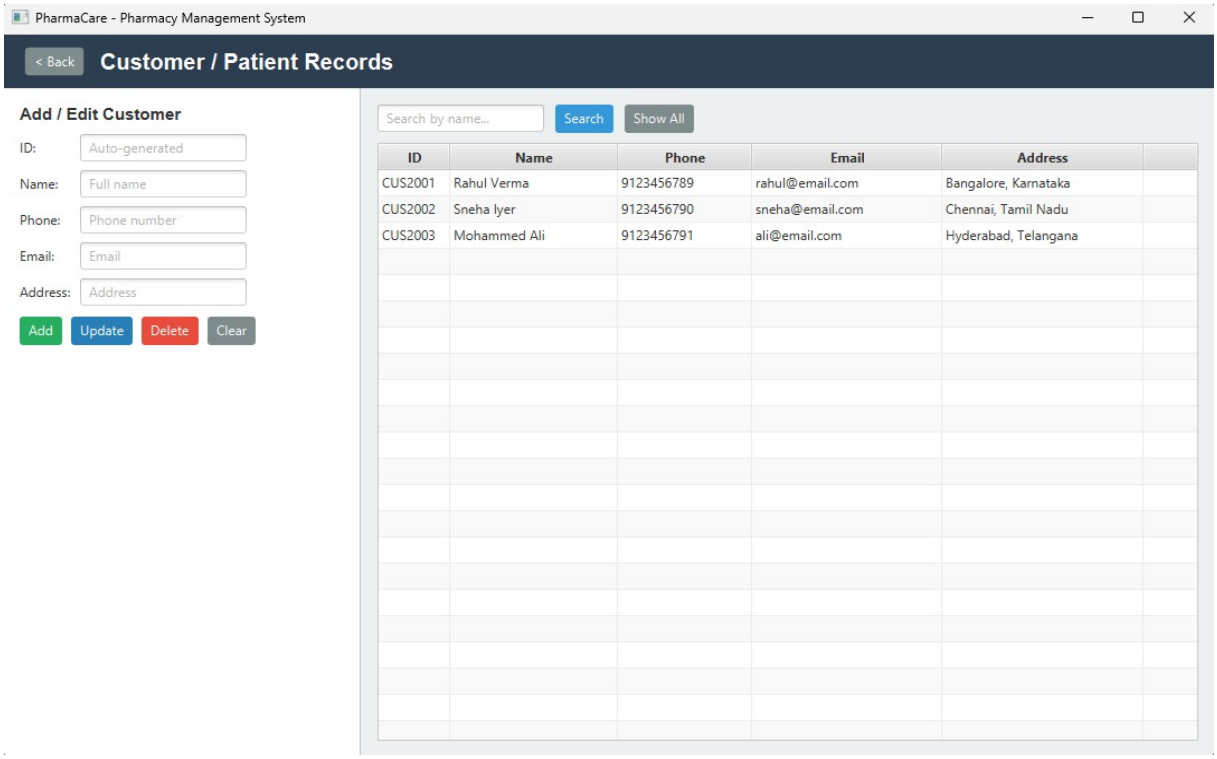
Sales and Billing Screen



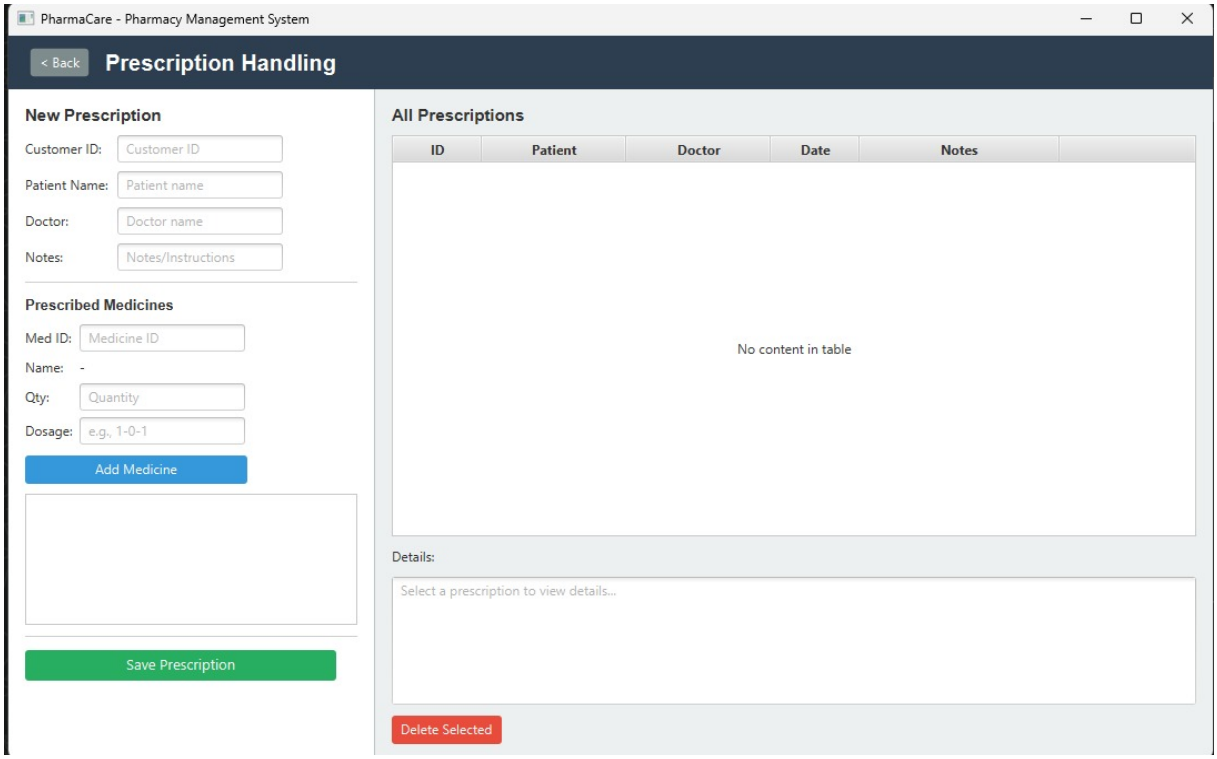
Supplier Management Screen



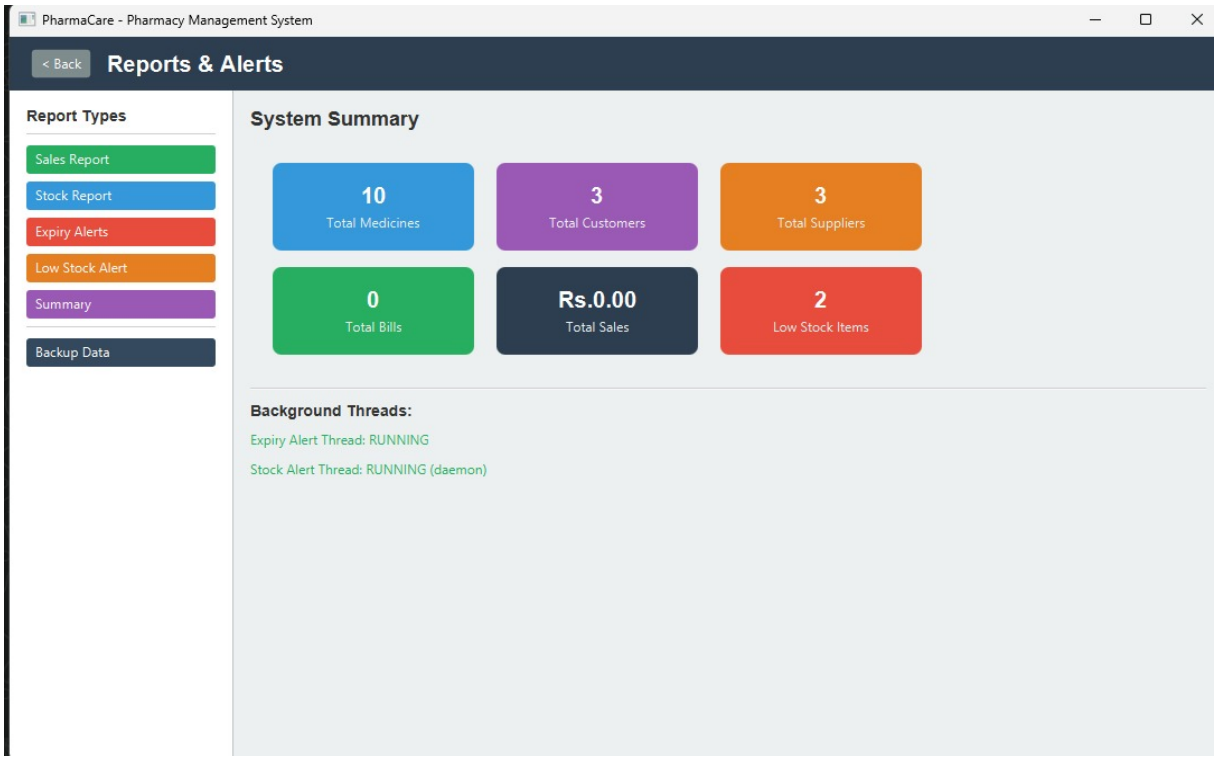
Customer Records Screen



Prescription Handling Screen



Reports and Alerts Screen


PharmaCare - Pharmacy Management System
< Back
Reports & Alerts
Report Types
Sales Report
Stock Report
Expiry Alerts
Low Stock Alert
Summary
Backup Data
System Summary
10
Total Medicines
3
Total Customers
3
Total Suppliers
0
Total Bills
Rs.0.00
Total Sales
2
Low Stock Items
Background Threads:
Expiry Alert Thread: RUNNING
Stock Alert Thread: RUNNING (daemon)

Conclusion

By the end of this project we had a fully working pharmacy application that we could actually demonstrate with real data. Looking back, the part that took the most thought wasn't the UI or even the threading - it was getting the class structure right. Once the model and service layers were cleanly separated, everything else fell into place a lot more naturally. Adding a new feature meant touching one service class and one screen, not rewriting things all over the codebase.

The threading part was genuinely useful to implement. Running expiry and stock checks in the background means the user never has to manually refresh anything - the system just tells them when something needs attention. Using BillingThread for invoice processing also showed us why Platform.runLater() matters: without it, UI updates from a background thread would just silently fail or cause exceptions.

On the academic side, this project ended up covering every OOP concept from the syllabus and then some. Encapsulation is in every model class. Inheritance shows up in the Thread subclasses and the Application extension. Polymorphism comes through with overloaded methods in Bill.java and the callback interfaces used in the threading classes. Abstraction is in those same interfaces. Generics in DataStore<T>. Collections, Serialization, File I/O, Enums, Wrapper classes - they're all genuinely used, not just added as decorations. That made the code feel like a real project rather than just a lab exercise.

Individual Contributions

Kenneth Martin, 240962047, Roll No. 8

- PharmacyApp.java
- LoginScreen.java
- DashboardScreen.java
- UserService.java
- User.java / UserRole.java

Sam Thomas, 240962242, Roll No. 18

- Medicine.java / MedicineCategory.java
- InventoryManager.java
- InventoryScreen.java
- SampleDataLoader.java

Mohammed Rayyan Sheik, 240962073, Roll No. 12

- BillingService.java
- BillingThread.java
- BillingScreen.java
- Bill.java / BillItem.java

Vedank Singh, 240962043, Roll No. 7

- DataStore.java
- IDGenerator.java
- CustomerService.java / SupplierService.java
- PrescriptionService.java
- ExpiryAlertThread.java / StockAlertThread.java
- BackupService.java

- ReportsScreen.java
- Supplier.java / Customer.java / Prescription.java / PrescriptionItem.java / BillItem.java

We split the work based on what each person was comfortable with, but we all ended up reviewing and testing each other's code anyway. OOP projects are inherently interconnected - Kenneth's login screen needed Sam's Medicine class to exist, Rayyan's billing needed Vedank's DataStore to work. So even though we had clear ownership of different parts, we were constantly in each other's code fixing import paths, adjusting method signatures, and debugging integration issues together. That back-and-forth is actually what made the final result consistent.

References

- 1. JavaFX Official Documentation: https://openjfx.io/
- 2. JavaFX Tutorials and Codes: https://www.geeksforgeeks.org/tag/javafx/
- 3. Java Serialization Guide: https://www.baeldung.com/java-serialization
- 4. Java Collections Framework: https://docs.oracle.com/javase/8/docs/technotes/guides/collections/overview.html
- 5. Java Multithreading: https://www.geeksforgeeks.org/multithreading-in-java/